claude-windows / d1586dfe-adae-4d40-a85d-ed0bb23738b8

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\d1586dfe-adae-4d40-a85d-ed0bb23738b8\subagents\agent-a0a70ddb1d7197b83.jsonl`
- SHA-256: `1667c8caa5596ccbb09c99ecc2f1eb2578a33c8646d61459a97bbc60121ebeee`
- Source modified: `2026-06-10T18:11:29+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `subagents`
- session_id: `d1586dfe-adae-4d40-a85d-ed0bb23738b8`

Transcript

1. user (2026-06-10T17:59:14.460Z)

You are working in an isolated git worktree of C:\Users\avidu\Projects\muneem (a Python 3.11+ project, src layout, package src/muneem, venv at the ORIGINAL repo path C:\Users\avidu\Projects\muneem\.venv ? use C:\Users\avidu\Projects\muneem\.venv\Scripts\python.exe to run tools; the editable install resolves `muneem` imports from src/, which exists identically in your worktree, but to be safe run pytest with the env var PYTHONPATH pointing at YOUR worktree's src dir so your changes are the ones under test. Verify which code is being tested before trusting results: e.g. temporarily add a failing assert in your worktree to confirm it fires).

MISSION: broaden the synthetic full-pipeline regression corpus (deep-review item D1). The project parses Indian bank statement PDFs with an arithmetic verification oracle. tests/synthpdf.py hand-rolls valid born-digital PDFs (stdlib only, real Helvetica metrics so right-aligned amount columns land where the coordinate clusterer expects). tests/test_regression_synth.py runs them through the REAL pipeline: extract (pdfminer) ? get_parser ? parse_and_insert ? assert reconciled. Today only HDFC savings + AU savings layouts are covered, single-page, 3 perfectly-aligned rows.

Read first: tests/synthpdf.py, tests/test_regression_synth.py, src/muneem/parsers/ (savings.py, axis.py, au.py, hdfc.py, engine.py, profiles.py, validation.py, clustering.py), src/muneem/cli.py (the ingest command + _ingest_bank_statement), src/muneem/statement_unlock.py + src/muneem/password_rules.py (the hybrid unlock), tests/test_cli.py for CLI test conventions, tests/conftest.py (autouse keychain isolation).

IMPORTANT recent contract changes you must honor (all on main):
- The savings trust oracle now ANCHORS a passing balance walk to the printed Opening/Closing Balance lines when present (savings.reconcile_oracle) ? your fixtures' first row must imply the printed opening and last row must equal the printed closing.
- parse_amount(signed=True) on balances: a "Dr" suffix = negative.
- Dates are normalized to ISO at the build boundary (parsers/dates.py).
- Axis CC persists with documents.status = "unverified" (Confidence.UNVERIFIED), and quarantines (stores nothing, "unreconciled") if any date-matched row's amount cell fails to parse.
- Parser routing requires letterhead-shaped evidence: "Axis Bank" + "Statement of Account"/"Account Statement" (title case, case-sensitive) for Axis savings; "Axis Bank" + "Credit Card Statement" for Axis CC; "au small finance bank" or "aubank.in" for AU; "hdfc bank"/"hdfcbank.com" + statement markers for HDFC.
- `txns list` tags any document status != reconciled.

DELIVERABLES (new tests in tests/test_regression_synth.py + helpers in tests/synthpdf.py):
1. **Axis savings born-digital fixture** routed via get_parser to AxisParser through the real pdfplumber path, reconciling + persisting (study AXIS_PROFILE in profiles.py for the column order/date format the engine expects; include "Axis Bank" + "Statement of Account" letterhead text and printed "Opening Balance"/"Closing Balance" lines consistent with the rows).
2. **Axis credit-card fixture** routed to AxisCreditCardParser: a 9+-column table (see axis._rows_from_cc_tables: date col 0 matching dd/mm/yyyy, particulars col 2, merchant category col 7, amount col 8 with "Dr"/"Cr" suffix) ? asserting rows persist with status "unverified".

NOTE: the CC parser uses page.extract_tables(), so the PDF needs ruled table lines OR you may construct this fixture at a different seam if extract_tables can't see a hand-rolled unruled table ? investigate what synthpdf can produce; if a true born-digital CC PDF through extract_tables is not feasible with reasonable effort, cover the CC path at the highest seam you CAN reach mechanically (document why in a comment), but try the PDF route first (pdfplumber extract_tables needs explicit lines: synthpdf would need to draw rects/lines ? PDF content streams can draw lines with simple operators like `x y m x y l S`; this is very doable in hand-rolled PDF).
3. **Multi-page savings statement** (HDFC or AU style): transaction rows split across 2 pages with the header repeated ? full pipeline must still reconcile + persist the combined set.
4. **Dropped-row-must-quarantine at the PDF level**: a statement whose printed rows are internally walk-consistent but missing a row vs the printed bookends (e.g. drop the first transaction; keep the printed Opening Balance of the full set) ? the pipeline must store NOTHING and record "unreconciled" (this reproduces the real Axis May-2025 class of failure at the PDF level and directly exercises the new bookend anchoring).
5. **Password-protected synthetic PDF through the real CLI**: encrypt a synthetic statement with pypdf (AES or RC4, pypdf is already a dependency; e.g. writer.encrypt("TEST1234")) and drive `muneem ingest <pdf> --doc-type bank_statement --db <tmp>` via typer CliRunner so the REAL hybrid unlock path runs (do NOT monkeypatch resolve_password; inject the candidate password through a legitimate seam: read statement_unlock.py/password_rules.py to find it ? e.g. the password_rules file location override or the keychain candidate cache via the conftest fake keychain store fixture `_isolate_keychain` ? seed the cache entry the way _remember writes it). Assert exit 0, rows persisted, status reconciled.

RULES: offline + deterministic (no network, no real data, no real bank names beyond the routing markers, fabricated amounts), all tests in the DEFAULT suite (no new markers), stdlib-only PDF generation (synthpdf style ? pypdf allowed only for encryption since it's already a dep). Keep synthpdf helpers pure and reusable. Match the repo's comment/docstring style (explanatory, no narration).

GATE + SHIP: from the worktree run: ruff check --fix, ruff format, mypy src, and the full pytest with coverage (.venv\Scripts\python.exe -m pytest -q --cov=muneem --cov-fail-under=80 with PYTHONPATH set to your worktree src; ALSO run plain pytest without coverage to be sure). All green. Then: git checkout -b test/synthetic-corpus-breadth, commit everything with a clear message ending in the trailer "Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>", push with -u origin, and create a PR with `gh pr create` (title "test: broaden the synthetic regression corpus (Axis savings/CC, multi-page, dropped-row, encrypted-PDF CLI)", body describing what each fixture proves, ending with the line "? Generated with [Claude Code](https://claude.com/claude-code)"). Do NOT merge the PR. If a deliverable proves genuinely infeasible after real attempts, ship the others and say exactly what blocked it in the PR body.

Your final message: the PR URL, which deliverables shipped/blocked, test counts, and any caveats. Raw facts, no pleasantries.

2. user (2026-06-10T17:59:20.359Z)

```
1       """Dependency-free, born-digital PDF generator for regression fixtures (stdlib only).
2
3       Layer 1 of the regression-CI plan (ROADMAP § "Planned ? Regression CI") needs **full-
pipeline**
4       fixtures ? real PDFs that go through pdfminer/pdfplumber, not synthetic word-lists that
bypass them.
5       Rather than add a dependency (reportlab etc.), we emit a minimal valid PDF with
**positioned
6       Helvetica text** (`Tm`/`Tj`), so it stays commercial-safe and CI-portable.
7
8       Coordinates: callers think in pdfplumber's top-down `top` (distance from the page top);
we convert
9       to PDF's bottom-up baseline. Absolute `top` need not be exact ? parsers key on
x-positions, same-row
10      grouping, and row order ? but words on one row share a baseline (one `top`) and rows are
separated.
11      Amount columns are **right-aligned** via real Helvetica metrics so their right edges
(`x1`) land in
12      the bands the HDFC clusterer / engine expect.
```

```
    """

    from __future__ import annotations

    from dataclasses import dataclass
    from pathlib import Path

    # Helvetica advance widths (per 1000 em). Exact for the chars that matter for right-
aligned amounts
    # (digits / comma / period / minus / space); a reasonable default covers labels (left-
aligned, so
    # their width is irrelevant to placement).
    _W = {
        " ": 278,
        "!": 278,
        '"': 355,
        "#": 556,
        "$": 556,
        "%": 889,
        "&": 667,
        "'": 191,
        "(": 333,
        ")": 333,
        "*": 389,
        "+": 584,
        ",": 278,
        "-": 333,
        ".": 278,
        "/": 278,
        "0": 556,
        "1": 556,
        "2": 556,
        "3": 556,
        "4": 556,
        "5": 556,
        "6": 556,
        "7": 556,
        "8": 556,
        "9": 556,
        ":": 278,
        ";": 278,
        "<": 584,
        "=": 584,
        ">": 584,
        "?": 556,
        "@": 1015,
    }
    _UPPER = 667  # rough average for A-Z
    _LOWER = 528  # rough average for a-z


    def helvetica_width(text: str, size: float) -> float:
        """Advance width of ``text`` in Helvetica at ``size`` points."""
        total = 0
        for ch in text:
            if ch in _W:
                total += _W[ch]
            elif ch.isupper():
                total += _UPPER
            else:
                total += _LOWER
        return total / 1000.0 * size


    @dataclass
```

```
    class Placement:
        """One text run: its LEFT edge ``x``, top-down ``top``, and the string."""

        x: float
        top: float
        text: str
        size: float = 9.0


    def at(x: float, top: float, text: str, size: float = 9.0) -> Placement:
        """Left-aligned run starting at ``x``."""
        return Placement(x=x, top=top, text=text, size=size)


    def right(x1: float, top: float, text: str, size: float = 9.0) -> Placement:
        """Right-aligned run whose right edge lands at ``x1`` (for amount columns)."""
        return Placement(x=x1 - helvetica_width(text, size), top=top, text=text, size=size)


    def _esc(text: str) -> str:
        return text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")


    def build_pdf(
        placements: list[Placement], path: Path | str, *, width: float = 595.0, height:
float = 842.0
    ) -> Path:
        """Write a single-page PDF with the given positioned text. Returns the path.

        Hand-rolls a minimal PDF (catalog ? pages ? page ? Helvetica font ? content stream)
with a
        byte-accurate xref table so pdfminer/pdfplumber parse it.
        """
        ops: list[str] = []
        for p in placements:
            y = height - p.top  # top-down -> PDF bottom-up baseline
            ops.append(f"BT /F1 {p.size:.2f} Tf 1 0 0 1 {p.x:.2f} {y:.2f} Tm
({_esc(p.text)}) Tj ET")
        content = ("\n".join(ops)).encode("latin-1")

        objects: list[bytes] = [
            b"<< /Type /Catalog /Pages 2 0 R >>",
            b"<< /Type /Pages /Kids [3 0 R] /Count 1 >>",
            (
                f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {width:.0f} {height:.0f}] "
                f"/Resources << /Font << /F1 4 0 R >> >> /Contents 5 0 R >>"
            ).encode("latin-1"),
            b"<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>",
            b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
b"\nendstream",
        ]

        out = bytearray(b"%PDF-1.4\n")
        offsets: list[int] = []
        for i, obj in enumerate(objects, start=1):
            offsets.append(len(out))
            out += f"{i} 0 obj\n".encode() + obj + b"\nendobj\n"

        xref_pos = len(out)
        count = len(objects) + 1
        out += f"xref\n0 {count}\n".encode()
        out += b"0000000000 65535 f \n"
        for off in offsets:
            out += f"{off:010d} 00000 n \n".encode()
        out += (
```

```
            b"trailer\n<< /Size " + str(count).encode() + b" /Root 1 0 R >>\n"
            b"startxref\n" + str(xref_pos).encode() + b"\n%%EOF"
        )

        out_path = Path(path)
        out_path.write_bytes(out)
        return out_path


    __all__ = ["Placement", "at", "right", "helvetica_width", "build_pdf"]

```

3. user (2026-06-10T17:59:21.119Z)

```
      """Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.

      Generates *real* PDFs (no committed statements, no secrets) and runs the production
pipeline ?
      ``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert a **reconciled**
parse. This
      covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the
synthetic
      word-list & table unit tests (which bypass pdfplumber) don't reach. See ROADMAP §
"Planned ?
      Regression CI for statement parsing" (Layer 1). Always-on, secret-free ? runs in cloud
CI.
      """

      from pathlib import Path

      from muneem import db as dbmod
      from muneem.extract import extract
      from muneem.parsers import get_parser
      from synthpdf import at, build_pdf, right


      def _hdfc_pdf(path: Path) -> Path:
          """HDFC layout: leading date x0?52, narration x0?111, three right-aligned amount
columns whose
          right edges land in the clusterer's bands (withdrawal <400, deposit <490, balance
?490), plus a
          SUMMARY block for the bookend. Three rows reconcile (opening 10,000 ? closing
10,000)."""
          p = [
              at(50, 60, "HDFC BANK"),
              at(50, 72, "Savings Account Details"),
              at(50, 84, "Account Number : 50100012345678"),
              # header ? must carry "Narration" + a "Closing" token so _table_body locates the
table
              at(72, 110, "Date"),
              at(175, 110, "Narration"),
              at(300, 110, "Withdrawals"),
              at(390, 110, "Deposits"),
              at(480, 110, "Closing"),
              at(515, 110, "Balance"),
          ]
          rows = [
              ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
              ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
              ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
          ]
          top = 130
          for d, narr, wd, dep, bal in rows:
              p += [
                  at(52, top, d),
```

```
                at(111, top, narr),
                right(358, top, wd),
                right(443, top, dep),
                right(561, top, bal),
            ]
            top += 15
        p += [
            at(50, 200, "SUMMARY"),
            at(50, 212, "Opening Balance Debit Amount Credit Amount Closing Balance"),
            at(50, 224, "10,000.00 2,000.00 2,000.00 10,000.00"),
            at(50, 236, "Debit Count Credit Count"),
            at(50, 248, "2 1"),
        ]
        return build_pdf(p, path)


    def _au_savings_pdf(path: Path) -> Path:
        """Savings layout (the shared savings.py engine path used by Axis/AU): a clean
        date | narration | debit | credit | balance table in well-separated columns ? coordinate
        clustering ? engine mapping ? per-row reconcile. Three rows that walk (opening 50,000)."""
        p = [
            at(50, 60, "AU Small Finance Bank"),
            at(50, 72, "Account Number : 1234567890123"),
            # header (clustering's date+money filter drops it; here only for realism / text markers)
            at(52, 100, "Date"),
            at(110, 100, "Transaction"),
            at(310, 100, "Withdrawal"),
            at(395, 100, "Deposit"),
            at(485, 100, "Balance"),
        ]
        rows = [
            ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
            ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
            ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
        ]
        top = 124
        for d, narr, debit, credit, bal in rows:
            line = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
            if debit:
                line.append(right(340, top, debit))
            if credit:
                line.append(right(420, top, credit))
            p += line
            top += 15
        return build_pdf(p, path)


    def _run_pipeline(pdf: Path) -> tuple[str, int, str]:
        """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back status."""
        ex = extract(pdf)
        parser = get_parser(ex.text)
        assert parser is not None, "no parser routed this synthetic statement"
        with dbmod.connect(":memory:") as conn:
            dbmod.init_db(conn)
            doc_id = dbmod.insert_document(conn, source="file", sha256=ex.sha256, status="parsed")
            n = parser.parse_and_insert(pdf, conn, doc_id)
            status = dbmod.get_document_by_sha(conn, ex.sha256)["status"]
        return type(parser).__name__, n, status


```

```
    def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
        name, n, status = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
        assert name == "HDFCParser"
        assert n == 3, f"expected 3 transactions, got {n}"
        assert status == "reconciled", f"expected reconciled, got {status}"


    def test_savings_layout_full_pipeline_reconciles(tmp_path):
        name, n, status = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
        assert name == "AUParser"
        assert n == 3, f"expected 3 transactions, got {n}"
        assert status == "reconciled", f"expected reconciled, got {status}"

```

4. user (2026-06-10T17:59:26.006Z)

```
      """Shared engine-driven savings-statement parsing (Axis, AU Bank, and future issuers).

      Every issuer whose statement is a transaction table follows the same recipe, so it lives
here once
      and each bank supplies only a :class:`~muneem.parsers.engine.Profile`:

      1. **Extraction proposes** candidate cell-matrices ? one per ``extract_tables()`` table,
plus a
         coordinate-clustered whole-statement matrix (for layouts ``extract_tables`` mis-
segments).
      2. The generic **engine** maps each candidate's columns onto the canonical concepts.
      3. **Reconciliation disposes:** a parse is trusted only if its per-row balance walks, or
? when a
         vintage has no per-row balance ? the totals reconcile to the printed
``opening``/``closing``.

      A statement splits its ledger across several extracted tables (page breaks), so we map
each table
      on its own, keep the rows the oracle endorses, and re-check the *combined* set (which
catches a
      cross-table balance break, i.e. a missing transaction). The result is **oracle-gated**:
a layout we
      can't yet read returns ``verified=False`` and the caller persists nothing ? never
mislabeled rows.

      No statement content is logged; only counts/booleans travel out.
      """

      from __future__ import annotations

      import re
      from dataclasses import dataclass, field
      from pathlib import Path

      import pdfplumber

      from .clustering import cluster_to_matrix
      from .concepts import ConceptRow, StatementConcepts
      from .dates import normalize_statement_date
      from .engine import Profile, map_table
      from .validation import (
          BALANCE_TOLERANCE,
          parse_amount,
          reconcile_running_balance,
          reconcile_total,
      )

      _MONEY_TOKEN = re.compile(r"^-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
      # Capture an optional trailing Cr/Dr marker: "Opening Balance: 5,000.00 Dr" is NEGATIVE
```

```
    # (overdrawn) and must anchor the oracle with its true sign.
    _OPENING_RE = re.compile(
        r"opening\s+balance[^\d\-]*(-?[\d,]+\.\d{2})(\s*(?:cr|dr))?", re.IGNORECASE
    )
    _CLOSING_RE = re.compile(
        r"closing\s+balance[^\d\-]*(-?[\d,]+\.\d{2})(\s*(?:cr|dr))?", re.IGNORECASE
    )


    def _summary_balance(m: re.Match | None) -> float | None:
        return parse_amount(f"{m.group(1)}{m.group(2) or ''}", signed=True) if m else None


    @dataclass
    class SavingsParse:
        """Outcome of a savings parse: concept rows + whether reconciliation endorsed
them."""

        account_number: str
        rows: list[ConceptRow] = field(default_factory=list)
        verified: bool = False


    def parse_statement_summary(text: str) -> StatementConcepts:
        """Read opening/closing balance from statement text (for the total-reconcile
oracle).

        Best-effort regex; ``None`` when not found ? the oracle then relies on the per-row
walk.
        """
        return StatementConcepts(
            opening_balance=_summary_balance(_OPENING_RE.search(text)),
            closing_balance=_summary_balance(_CLOSING_RE.search(text)),
        )


    def txn_line_predicate(profile: Profile):
        """A line is transaction-like if it carries one of the profile's dates *and* a money
token.

        Used to restrict coordinate clustering to the transaction band (drops headers /
summaries /
        address blocks that would otherwise wreck column inference).
        """
        patterns = profile.date_patterns

        def keep(texts: list[str]) -> bool:
            has_date = any(any(p.match(t) for p in patterns) for t in texts)
            return has_date and any(_MONEY_TOKEN.match(t) for t in texts)

        return keep


    def reconcile_oracle(opening: float | None, closing: float | None, tol: float =
BALANCE_TOLERANCE):
        """Trust a parse only when the arithmetic AND every available printed bookend agree.

        The per-row walk proves *internal* consistency, but it cannot see rows missing at
the
        statement's **edges** ? dropping the first or last transaction leaves a perfectly
        consistent chain. So a passing walk is additionally anchored to the printed
        opening/closing balance whenever the summary supplied them: the first row must imply
        the printed opening (``balance ? credit + debit``), the last row must equal the
printed
        closing. Without a usable walk, both bookends + the total identity carry the
```

```
verdict.
98
99          This is the **trust** oracle for a whole-statement row set. Per-table *selection*
during
100         multi-page mapping uses :func:`_selection_oracle` instead ? a page fragment can't
span
101         the bookends.
102         """
103
104         def check(rows: list[ConceptRow]) -> bool:
105             if not rows:
106                 return False
107             if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
108                 first, last = rows[0], rows[-1]
109                 if opening is not None and first.balance is not None:
110                     implied = first.balance - (first.credit or 0.0) + (first.debit or 0.0)
111                     if abs(implied - opening) > tol:
112                         return False
113                 if closing is not None and last.balance is not None:
114                     if abs(last.balance - closing) > tol:
115                         return False
116                 return True
117             if opening is not None and closing is not None:
118                 return reconcile_total(rows, opening, closing)
119             return False
120
121         return check
122
123
124     def _selection_oracle(opening: float | None, closing: float | None):
125         """Endorse a candidate column-mapping for ONE table (possibly a page fragment).
126
127         A fragment's walk proves the mapping is right even though the fragment can't span
the
128         printed bookends, so no edge anchoring here ? the combined row set is re-judged by
the
129         strict :func:`reconcile_oracle` before anything is trusted.
130         """
131
132         def check(rows: list[ConceptRow]) -> bool:
133             if reconcile_running_balance([(r.debit, r.credit, r.balance) for r in rows]).ok:
134                 return True
135             if rows and opening is not None and closing is not None:
136                 return reconcile_total(rows, opening, closing)
137             return False
138
139         return check
140
141
142     def _reconstruct_by_balance_delta(
143         matrix: list[list[str]], profile: Profile, opening: float | None, tol: float =
BALANCE_TOLERANCE
144     ) -> list[ConceptRow]:
145         """Recover debit/credit from the running balance when columns won't split cleanly.
146
147         Needs only three things per row ? a date, the transaction amount, and the running
balance ? so
148         it tolerates messy column inference (a sliver column, an interleaved value-date
column, or a
149         single merged withdrawal/deposit column). Direction is the sign of the balance
change; the
150         caller's reconciliation then **cross-checks** that the amount equals |?balance| on
every row, so
151         a misread or a dropped row still fails (it is *not* circular). Needs the printed
opening balance
```

```
        to fix the first row's direction; returns ``[]`` if it can't form a clean series.
        """
        patterns = profile.date_patterns or (re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),)
        rows = [[(str(c).strip() if c else "") for c in r] for r in (matrix or []) if r]
        if not rows or opening is None:
            return []
        ncols = max(len(r) for r in rows)

        def is_date(s: str) -> bool:
            return any(p.match(s) for p in patterns)

        def money(s: str) -> float | None:
            return parse_amount(s) if _MONEY_TOKEN.match(s) else None

        def balance_money(s: str) -> float | None:
            # Balance cells honour a Cr/Dr suffix as a SIGN (Dr = overdrawn = negative).
            return parse_amount(s, signed=True) if _MONEY_TOKEN.match(s) else None

        # date column = the leftmost of the (first few) columns with the most date cells
        date_counts: dict[int, int] = {}
        for r in rows:
            for c in range(min(ncols, 3)):
                if c < len(r) and is_date(r[c]):
                    date_counts[c] = date_counts.get(c, 0) + 1
        if not date_counts:
            return []
        top = max(date_counts.values())
        date_col = min(c for c, n in date_counts.items() if n == top)

        txn_rows = [r for r in rows if date_col < len(r) and is_date(r[date_col])]
        if len(txn_rows) < 2:
            return []

        # A money column's money cells must outnumber its date cells ? so a sparsely-filled
deposit
        # column still counts, but an interleaved value-date column (mostly dates) does not.
The
        # running balance is the rightmost money column; the rest are the flow
(withdrawal/deposit).
        money_cols = []
        for c in range(ncols):
            m = sum(1 for r in txn_rows if c < len(r) and money(r[c]) is not None)
            d = sum(1 for r in txn_rows if c < len(r) and is_date(r[c]))
            if m >= 1 and m > d:
                money_cols.append(c)
        if len(money_cols) < 2:
            return []
        balance_col = max(money_cols)
        flow_cols = [c for c in money_cols if c != balance_col]

        text_cols = [c for c in range(ncols) if c != date_col and c not in money_cols]
        narr_col = (
            max(text_cols, key=lambda c: sum(len(r[c]) for r in txn_rows if c < len(r)))
            if text_cols
            else None
        )

        out: list[ConceptRow] = []
        prev = opening
        for r in txn_rows:
            bal = balance_money(r[balance_col]) if balance_col < len(r) else None
            if bal is None:
                return []  # a running balance on every row is required for the delta
            amt = next((money(r[c]) for c in flow_cols if c < len(r) and money(r[c]) is not
None), None)
```

```
            debit = credit = None
            if amt is not None:
                if bal < prev - tol:
                    debit = amt
                elif bal > prev + tol:
                    credit = amt
                else:
                    debit = amt  # zero net with an amount present ? reconciliation will
flag it
            out.append(
                ConceptRow(
                    txn_date=normalize_statement_date(r[date_col]) or "",
                    narration=(r[narr_col] if narr_col is not None and narr_col < len(r)
else ""),
                    debit=debit,
                    credit=credit,
                    balance=bal,
                )
            )
            prev = bal
        return out


    def parse_savings(pdf_path: Path, profile: Profile, password: str | None = None) ->
SavingsParse:
        """Parse a savings statement into concept rows + a reconciliation verdict, per the
profile.

        Per-table mapping first (each extracted table mapped on its own; oracle-endorsed
rows combined
        and re-checked), then a coordinate-clustering fallback. Returns ``verified=False``
if neither
        reconciles, so the caller can refuse to persist.
        """
        with pdfplumber.open(pdf_path, password=password) as pdf:
            words_by_page = [
                page.extract_words(use_text_flow=False, keep_blank_chars=False) for page in
pdf.pages
            ]
            tables = [t for page in pdf.pages for t in (page.extract_tables() or [])]
            head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))

        account_number = profile.first_account(head) or "UNKNOWN"
        summary = parse_statement_summary(head)
        trust = reconcile_oracle(summary.opening_balance, summary.closing_balance)
        select = _selection_oracle(summary.opening_balance, summary.closing_balance)

        confirmed: list[ConceptRow] = []
        for table in tables:
            res = map_table(table, profile, oracle=select)  # fragment-level mapping
selection
            if res and res.verified:
                confirmed.extend(res.rows)
        if confirmed and trust(confirmed):  # the combined set must pass the bookend-
anchored oracle
            return SavingsParse(account_number, confirmed, True)

        # The clustered matrix and the delta reconstruction span the whole statement, so the
strict
        # bookend-anchored oracle judges them directly.
        matrix = cluster_to_matrix(words_by_page, keep_line=txn_line_predicate(profile))
        clustered = map_table(matrix, profile, oracle=trust)
        if clustered and clustered.verified:
            return SavingsParse(account_number, clustered.rows, True)
```

```
267
268         # 3) balance-delta reconstruction ? recovers vintages whose columns don't split into
a clean
269         #    debit/credit (merged amount column, interleaved value-date, sliver columns):
derive the
270         #    direction from the running balance and let the oracle cross-check amount ==
|?balance|.
271         delta = _reconstruct_by_balance_delta(matrix, profile, summary.opening_balance)
272         if delta and trust(delta):
273             return SavingsParse(account_number, delta, True)
274
275         fallback = confirmed or (clustered.rows if clustered else [])
276         return SavingsParse(account_number, fallback, False)
277
278
279     __all__ = [
280         "SavingsParse",
281         "parse_savings",
282         "parse_statement_summary",
283         "txn_line_predicate",
284         "reconcile_oracle",
285     ]
286
```

5. user (2026-06-10T17:59:26.665Z)

```
1       """Per-issuer profiles ? bank knowledge expressed as **data, not code** (architecture
§6).
2
3       A profile carries the priors the generic engine needs to disambiguate one issuer: extra
4       concept?label aliases, the accepted date shape, a positional fallback when there's no
header,
5       table anchors, account-number patterns, quirks, and (parked) the password rule. Adding
an issuer is
6       ideally *adding a profile*, not writing a parser.
7
8       `HDFC_PROFILE` is data consumed by HDFC's parser, whose *extraction* stays bespoke
coordinate-
9       clustering ? the documented exception where `extract_tables()` fails on a semi-ruled
table (see the
10      `hdfc-parser-tech-decision` memory). The Axis and AU Bank profiles land with their
engine-driven
11      parsers (coordinate clustering ? generic mapping ? total reconcile).
12      """
13
14      from __future__ import annotations
15
16      import re
17
18      from .engine import Profile
19
20      AXIS_PROFILE = Profile(
21          name="axis",
22          aliases={
23              # Real Axis savings header: Tran/Txn Date | Transaction | Withdrawals | Deposits
| Balance |
24              # Other Information. "Transaction" is the narration column (not the date).
25              "narration": ("transaction", "particulars"),
26              "debit": ("withdrawals", "withdrawal", "debit"),
27              "credit": ("deposits", "deposit", "credit"),
28              "balance": ("balance",),
29          },
30          date_patterns=(re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),),  # dd-mm-yyyy or
dd/mm/yyyy
31          # Layout / column-count varies between vintages, so there is NO reliable
```

```
column_order ? the
32          # generic content search + total reconcile do the work. Broadened account pattern:
the old
33          # `Account No:` regex missed real statements.
34          account_patterns=(
35              re.compile(r"Account\s*(?:No|Number)\s*[:.\-]?\s*(\d{6,})", re.IGNORECASE),
36              re.compile(r"A/c\s*No\.?\s*[:\s]*(\d{6,})", re.IGNORECASE),
37          ),
38          quirks=frozenset({"layout-varies", "other-information-column", "balance-optional"}),
39      )
40
41      AUBANK_PROFILE = Profile(
42          name="aubank",
43          aliases={
44              "narration": (
45                  "transaction",
46                  "particulars",
47                  "description",
48                  "narration",
49                  "transaction details",
50              ),
51              "debit": ("withdrawal", "withdrawals", "debit", "withdrawal amount", "dr"),
52              "credit": ("deposit", "deposits", "credit", "deposit amount", "cr"),
53              "balance": ("balance", "closing balance", "running balance"),
54          },
55          # AU vintages use dd-mm-yyyy / dd/mm/yyyy or dd-Mon-yyyy; verified on the real
statements.
56          date_patterns=(
57              re.compile(r"^\d{2}[-/]\d{2}[-/]\d{4}$"),
58              re.compile(r"^\d{2}-[A-Za-z]{3}-\d{4}$"),
59          ),
60          account_patterns=(
61              re.compile(r"Account\s*(?:No|Number)\s*[:.\-]?\s*(\d{6,})", re.IGNORECASE),
62              re.compile(r"A/c\s*No\.?\s*[:\s]*(\d{6,})", re.IGNORECASE),
63          ),
64          quirks=frozenset({"engine-driven"}),
65      )
66
67      HDFC_PROFILE = Profile(
68          name="hdfc",
69          aliases={
70              "debit": ("withdrawal", "withdrawals"),
71              "credit": ("deposit", "deposits"),
72              "balance": ("closing balance",),
73          },
74          date_patterns=(re.compile(r"^\d{2}/\d{2}/\d{4}$"),),  # dd/mm/yyyy
75          table_anchors=("Narration", "Closing Balance"),
76          account_patterns=(re.compile(r"Account\s*Number\s*[:.]?\s*(\d{6,})",
re.IGNORECASE),),
77          # HDFC's transaction table is semi-ruled; extract_tables() is unreliable on it, so
HDFC extracts
78          # by word-position clustering rather than through the generic cell-matrix engine.
79          quirks=frozenset({"semi-ruled-table", "coordinate-clustering"}),
80      )
81
82      __all__ = ["AXIS_PROFILE", "AUBANK_PROFILE", "HDFC_PROFILE"]
83
```

6. user (2026-06-10T17:59:33.223Z)

```
1       """Axis Bank statement parsers: savings/transaction account + credit card.
2
3       **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
table +
4       coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
```

```
gated** so a
5       vintage it can't read stores nothing (``unreconciled``) rather than mislabeled rows.
Axis specifics
6       live entirely in ``AXIS_PROFILE``; this replaced the old fixed-column ``extract_tables``
mapping.
7
8       Credit-card statements carry no running balance, so ``AxisCreditCardParser`` can't
reconcile
9       (parse/persist split + count only). No statement content is logged here.
10      """
11
12      from __future__ import annotations
13
14      import re
15      import sqlite3
16      from dataclasses import dataclass
17      from pathlib import Path
18
19      import pdfplumber
20
21      from muneem import __version__
22      from muneem import db as dbmod
23      from muneem.errors import ParseError
24      from muneem.redaction import redact_filename
25
26      from . import register_parser
27      from .base import StatementParser
28      from .dates import normalize_statement_date
29      from .profiles import AXIS_PROFILE
30      from .savings import SavingsParse, parse_savings, parse_statement_summary,
txn_line_predicate
31      from .validation import Confidence, parse_amount
32
33      _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}")  # Axis credit card: dd/mm/yyyy
34      _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*+\d{4})")]
35
36
37      def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
38          for rx in patterns:
39              m = rx.search(text)
40              if m:
41                  return m.group(1).strip()
42          return None
43
44
45      # --- Savings / transaction account (concept core)
----------------------------------------------
46
47
48      # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
49      AxisSavings = SavingsParse
50      parse_axis_summary = parse_statement_summary
51      _is_txn_line = txn_line_predicate(AXIS_PROFILE)
52
53
54      def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
55          """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
56          return parse_savings(pdf_path, AXIS_PROFILE, password)
57
58
59      @register_parser
60      class AxisParser(StatementParser):
61          def can_handle(self, text: str, sender: str | None = None, subject: str | None =
```

```
None) -> bool:
            if sender and "axisbank.com" in sender.lower():
                return True
            t = text or ""
            return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)

        def parse_and_insert(
            self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
        ) -> int:
            """Persist a savings parse **only if it reconciles**; otherwise store nothing.

            Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
            parser can't yet read surfaces loudly instead of saving mislabeled rows.
            """
            try:
                parsed = parse_axis_savings(pdf_path, password)
            except ParseError:
                raise
            except Exception as exc:
                # Filename only ? never statement text; original cause kept via `from`.
                name = redact_filename(pdf_path)
                raise ParseError(f"failed to parse Axis statement {name}") from exc

            if not parsed.verified:
                dbmod.record_parse(
                    conn,
                    doc_id,
                    confidence=Confidence.UNRECONCILED.value,
                    parser="AxisParser",
                    parser_version=__version__,
                )
                return 0

            for r in parsed.rows:
                dbmod.insert_axis_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=parsed.account_number,
                    txn_date=r.txn_date,
                    value_date=r.value_date or "",
                    particulars=r.narration,
                    debit=r.debit,
                    credit=r.credit,
                    balance=r.balance,
                )
            dbmod.record_parse(
                conn,
                doc_id,
                confidence=Confidence.RECONCILED.value,
                parser="AxisParser",
                parser_version=__version__,
            )
            return len(parsed.rows)


    # --- Credit card (no running balance; count only)
-------------------------------------------


    @dataclass
    class AxisCCTxn:
        txn_date: str
```

```
122         particulars: str
123         merchant_category: str
124         amount: float
125         type: str  # "Cr" or "Dr"
126 
127 
128     def _rows_from_cc_tables(tables: list) -> tuple[list[AxisCCTxn], int]:
129         """Build credit-card transactions from extracted tables. Pure: no PDF, no DB.
130 
131         Returns ``(rows, bad_amounts)`` ? ``bad_amounts`` counts transactional rows (date-
matched)
132         whose amount cell did not parse. With no balance walk to catch a misread, a silent
``0.0``
133         coercion would persist a wrong amount as if real; the caller must treat any bad
amount as
134         a suspect parse.
135         """
136         rows: list[AxisCCTxn] = []
137         bad_amounts = 0
138         for table in tables or []:
139             for row in table or []:
140                 if not row or len(row) < 9:
141                     continue
142                 date_str = str(row[0]).strip() if row[0] else ""
143                 if not _CC_DATE_RE.match(date_str):
144                     continue
145                 amount_str = str(row[8]).strip() if row[8] else ""
146                 is_cr = "Cr" in amount_str
147                 amount = parse_amount(re.sub(r"[^\d.]", "", amount_str))
148                 if amount is None:
149                     bad_amounts += 1
150                     continue
151                 rows.append(
152                     AxisCCTxn(
153                         txn_date=date_str,
154                         particulars=str(row[2]).strip() if row[2] else "",
155                         merchant_category=str(row[7]).strip() if row[7] else "",
156                         amount=amount,
157                         type="Cr" if is_cr else "Dr",
158                     )
159                 )
160         return rows, bad_amounts
161 
162 
163     def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn], int]:
164         """Open an Axis credit-card statement; return (card_number, transactions,
bad_amounts)."""
165         card_number = "UNKNOWN"
166         rows: list[AxisCCTxn] = []
167         bad_amounts = 0
168         with pdfplumber.open(pdf_path, password=password) as pdf:
169             head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
170             card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
171             for page in pdf.pages:
172                 page_rows, page_bad = _rows_from_cc_tables(page.extract_tables())
173                 rows.extend(page_rows)
174                 bad_amounts += page_bad
175         return card_number, rows, bad_amounts
176 
177 
178     @register_parser
179     class AxisCreditCardParser(StatementParser):
180         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
```

```
None) -> bool:
181             return "Axis Bank" in text and "Credit Card Statement" in text
182
183         def parse_and_insert(
184             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
185         ) -> int:
186             """Persist CC rows with an explicit ``unverified`` verdict ? never as silently
fine.
187
188             A credit-card bill has no running balance, so no arithmetic oracle can endorse
the
189             parse (Confidence.UNVERIFIED, surfaced by every listing). What CAN be checked is
190             checked: a transactional row whose amount cell does not parse means the layout
read
191             is suspect ? the parse is quarantined (store nothing, ``unreconciled``) rather
than
192             persisting a wrong amount the math could never catch.
193             """
194             try:
195                 card_number, rows, bad_amounts = parse_axis_cc(pdf_path, password)
196             except ParseError:
197                 raise
198             except Exception as exc:
199                 raise ParseError(
200                     f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
201                 ) from exc
202             if bad_amounts > 0:
203                 dbmod.record_parse(
204                     conn,
205                     doc_id,
206                     confidence=Confidence.UNRECONCILED.value,
207                     parser="AxisCreditCardParser",
208                     parser_version=__version__,
209                 )
210                 return 0
211             for t in rows:
212                 dbmod.insert_axis_cc_transaction(
213                     conn,
214                     document_id=doc_id,
215                     account_number=card_number,
216                     txn_date=normalize_statement_date(t.txn_date),
217                     particulars=t.particulars,
218                     merchant_category=t.merchant_category,
219                     amount=t.amount,
220                     type=t.type,
221                 )
222             dbmod.record_parse(
223                 conn,
224                 doc_id,
225                 confidence=Confidence.UNVERIFIED.value,
226                 parser="AxisCreditCardParser",
227                 parser_version=__version__,
228             )
229             return len(rows)
230
```

7. user (2026-06-10T17:59:33.535Z)

```
1       """The generic concept-mapping engine ? columns in, concepts out, arithmetic decides.
2
3       This is the *generic-first* half of the concept-core architecture (see
4       [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §4, §6).
5       Bank-agnostic code that takes a **cell-matrix** (the shape `pdfplumber.extract_tables()`
returns,
```

```
6       or what a coordinate-clustering extractor emits) and maps its columns onto the canonical
concepts
7       via a cheap-first ladder, with a profile supplying priors:
8
9       * **L0 ? header lexicon.** Match header-row labels against the profile's concept?label
aliases.
10      * **profile-order.** If there is no header, fall back to the profile's known column
order (a prior).
11      * **L1/L2 ? content + search.** Classify columns by content (dates, money), then
enumerate the
12        plausible amount-column ? {debit, credit, balance} assignments and **keep the one the
13        reconciliation oracle endorses** ? the self-identifying balance column, found by
*which mapping
14        makes the math work* rather than by its label.
15
16      The engine is **family-agnostic**: it is parameterized by a :class:`FamilySpec` (the
target concept
17      names, which are date/amount/balance, and a typed-row builder) and an injected `oracle`
18      (`rows -> bool`). Only the *bank* family ships today; an instrument family
(equity/MF/ETF/bond ? see
19      the project scope) would reuse this control flow with its own concepts + oracle. Nothing
here logs a
20      value; provenance is the winning layer + column map.
21      """
22
23      from __future__ import annotations
24
25      import itertools
26      import re
27      from collections.abc import Callable, Sequence
28      from dataclasses import dataclass, field
29
30      from .concepts import ConceptRow
31      from .dates import normalize_statement_date
32      from .validation import parse_amount
33
34      Cell = object  # a table cell: str | None (we coerce defensively)
35      Row = Sequence[Cell]
36
37      # Generic Indian-statement date shapes; a profile can pin its own to avoid false
positives.
38      _GENERIC_DATE_PATTERNS: tuple[re.Pattern, ...] = (
39          re.compile(r"^\d{2}[/-]\d{2}[/-]\d{4}$"),  # dd/mm/yyyy or dd-mm-yyyy
40          re.compile(r"^\d{2}[- ][A-Za-z]{3}[- ]\d{2,4}$"),  # dd-Mon-yyyy / dd Mon yy
41      )
42      # "Money" requires a decimal point so bare integers (e.g. a long reference number) are
NOT mistaken
43      # for an amount column during content classification.
44      _MONEY_RE = re.compile(r"(?i)^-?[\d,]+\.\d{1,2}(?:\s*(?:cr|dr))?$")
45      _BALANCE_MARKERS = ("opening balance", "closing balance", "balance b/f", "balance c/f")
46
47      # The generic bank lexicon. Profiles extend/override per issuer; keys are concept names,
values are
48      # lower-cased header labels matched exactly (not by substring) during L0.
49      _BANK_ALIASES: dict[str, tuple[str, ...]] = {
50          "txn_date": ("date", "txn date", "tran date", "transaction date", "posting date"),
51          "value_date": ("value date", "val date", "value dt"),
52          "narration": (
53              "narration",
54              "particulars",
55              "description",
56              "transaction details",
57              "details",
58              "remarks",
59              "transaction",
```

```
            "transaction remarks",
        ),
        "reference": (
            "ref",
            "reference",
            "ref no",
            "reference no",
            "cheque no",
            "chq no",
            "instrument no",
        ),
        "debit": ("debit", "withdrawal", "withdrawals", "dr", "paid out", "debit amount"),
        "credit": ("credit", "deposit", "deposits", "cr", "paid in", "credit amount"),
        "balance": (
            "balance",
            "closing balance",
            "running balance",
            "closing bal",
            "available balance",
        ),
    }


    def _text(cell: Cell) -> str:
        return "" if cell is None else str(cell).strip()


    def _norm(label: str) -> str:
        return re.sub(r"\s+", " ", label.strip().lower())


    def _matches_date(text: str, patterns: Sequence[re.Pattern]) -> bool:
        return any(p.match(text) for p in patterns)


    def _looks_like_money(text: str) -> bool:
        return bool(_MONEY_RE.match(text))


    @dataclass
    class FamilySpec:
        """A concept *family*: the target vocabulary + how to build typed rows from a column
mapping.

        The engine's L0/L1/L2 logic is written against this spec rather than against bank
concepts, so a
        future instrument family (with its own concepts + reconciliation oracle) reuses the
same engine.
        ``build`` turns per-row ``{concept: raw_cell}`` dicts into typed rows the oracle
understands.
        """

        name: str
        concepts: tuple[str, ...]
        date_concepts: tuple[str, ...]  # in order: the first is the primary (row-gating)
date
        amount_concepts: tuple[str, ...]  # money columns; the non-balance ones are the
"flows"
        balance_concept: str | None  # the running-balance concept, if the family has one
        text_concepts: tuple[str, ...]  # free-text columns; the first gets the widest text
column
        default_aliases: dict[str, tuple[str, ...]]
        build: Callable[[list[dict[str, str]]], list]


```

```
    def _build_bank_rows(mapped: list[dict[str, str]]) -> list[ConceptRow]:
        """Turn mapped cells into :class:`ConceptRow`s (amounts parsed; absent stays
``None``).

        Dates are canonicalized to ISO-8601 here (day-first Indian formats; unrecognized
tokens
        pass through verbatim) ? arithmetic cannot catch a date error, so at least the
stored
        form is a validated calendar date.
        """
        rows: list[ConceptRow] = []
        for m in mapped:
            rows.append(
                ConceptRow(
                    txn_date=normalize_statement_date(_text(m.get("txn_date"))) or "",
                    narration=_text(m.get("narration")),
                    debit=parse_amount(m.get("debit")),
                    credit=parse_amount(m.get("credit")),
                    # signed: a "Dr" suffix on a BALANCE means overdrawn (negative) ? see
parse_amount
                    balance=parse_amount(m.get("balance"), signed=True),
                    value_date=normalize_statement_date(_text(m.get("value_date")) or None),
                    reference=_text(m.get("reference")) or None,
                )
            )
        return rows


    BANK_FAMILY = FamilySpec(
        name="bank",
        concepts=("txn_date", "value_date", "narration", "reference", "debit", "credit",
"balance"),
        date_concepts=("txn_date", "value_date"),
        amount_concepts=("debit", "credit", "balance"),
        balance_concept="balance",
        text_concepts=("narration", "reference"),
        default_aliases=_BANK_ALIASES,
        build=_build_bank_rows,
    )


    @dataclass
    class MappingResult:
        """The outcome of mapping one table: typed rows + how we got there (provenance)."""

        rows: list  # typed rows from family.build (e.g. list[ConceptRow])
        mapped: list[
            dict[str, str]
        ]  # per-row {concept: raw cell}, incl. any profile-declared extra columns
        column_map: dict[str, int]  # the winning concept -> column-index assignment
        layer: str  # "L0" / "profile-order" / "L2" ? which rung produced it
        verified: bool  # did the oracle endorse it? (False if no oracle was supplied)


    @dataclass
    class Profile:
        """Declarative per-issuer knowledge ? *data, not code* (architecture §6).

        Generic detectors run first; the profile supplies priors: extra concept?label
aliases, the
        accepted date shape, a positional fallback when there's no header, table anchors,
account-number
        patterns, quirks, and (placeholder, wiring parked) the password rule.
        """

```

```
        name: str
        aliases: dict[str, tuple[str, ...]] = field(default_factory=dict)
        date_patterns: tuple[re.Pattern, ...] = ()
        column_order: tuple[str, ...] | None = None
        table_anchors: tuple[str, ...] = ()
        account_patterns: tuple[re.Pattern, ...] = ()
        quirks: frozenset[str] = frozenset()
        password_rule: str | None = None  # data only; sourcing is parked (DESIGN §8.2)

        def first_account(self, text: str) -> str | None:
            for rx in self.account_patterns:
                m = rx.search(text)
                if m:
                    return m.group(1).strip()
            return None


    def _alias_index(profile: Profile, family: FamilySpec) -> dict[str, str]:
        """Build ``normalized-label -> concept`` (profile entries override family
defaults)."""
        index: dict[str, str] = {}
        for concept, labels in family.default_aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        for concept, labels in profile.aliases.items():
            for label in labels:
                index[_norm(label)] = concept
        return index


    def _date_patterns(profile: Profile) -> tuple[re.Pattern, ...]:
        return profile.date_patterns or _GENERIC_DATE_PATTERNS


    def _detect_header(
        rows: list[list[Cell]], alias_index: dict[str, str]
    ) -> tuple[int | None, dict[str, int]]:
        """Find the header row (most exact alias hits, >=2) and its concept->column map."""
        best_idx: int | None = None
        best_map: dict[str, int] = {}
        for i, row in enumerate(rows):
            cmap: dict[str, int] = {}
            for col, cell in enumerate(row):
                concept = alias_index.get(_norm(_text(cell)))
                if concept and concept not in cmap:
                    cmap[concept] = col
            if len(cmap) > len(best_map):
                best_map, best_idx = cmap, i
        if len(best_map) < 2:
            return None, {}
        return best_idx, best_map


    def _is_balance_marker(row: Row) -> bool:
        return any(any(mk in _norm(_text(c)) for mk in _BALANCE_MARKERS) for c in row)


    def _is_transaction_row(row: Row, date_col: int | None, patterns: Sequence[re.Pattern])
-> bool:
        if date_col is not None:
            cell = row[date_col] if date_col < len(row) else None
            return _matches_date(_text(cell), patterns)
        return any(_matches_date(_text(c), patterns) for c in row)


```

```
    def _classify_columns(
        data: list[list[Cell]], ncols: int, patterns: Sequence[re.Pattern]
    ) -> dict[int, str]:
        """Tag each column date / amount / text / empty by majority content."""
        kinds: dict[int, str] = {}
        for c in range(ncols):
            cells = [_text(r[c]) for r in data if c < len(r) and _text(r[c])]
            if not cells:
                kinds[c] = "empty"
                continue
            thresh = max(1, int(len(cells) * 0.6))
            if sum(1 for x in cells if _matches_date(x, patterns)) >= thresh:
                kinds[c] = "date"
            elif sum(1 for x in cells if _looks_like_money(x)) >= thresh:
                kinds[c] = "amount"
            else:
                kinds[c] = "text"
        return kinds


    def _base_map(
        col_kinds: dict[int, str], data: list[list[Cell]], family: FamilySpec
    ) -> dict[str, int]:
        """The unambiguous part of a content (L1) mapping: date columns + the primary text
column.

        Family-agnostic: date columns fill ``family.date_concepts`` in order; the widest
text column
        goes to the family's first text concept (if any). Amount columns are left to the
search.
        """
        base: dict[str, int] = {}
        date_cols = [c for c, k in sorted(col_kinds.items()) if k == "date"]
        text_cols = [c for c, k in sorted(col_kinds.items()) if k == "text"]
        for concept, col in zip(family.date_concepts, date_cols, strict=False):
            base[concept] = col
        if text_cols and family.text_concepts:
            base[family.text_concepts[0]] = max(
                text_cols, key=lambda c: sum(len(_text(r[c])) for r in data if c < len(r))
            )
        return base


    def _search_candidates(
        base: dict[str, int], col_kinds: dict[int, str], family: FamilySpec
    ) -> list[tuple[str, dict[str, int]]]:
        """Enumerate plausible amount-column ? concept assignments (the L2 search space).

        Family-agnostic: the "flows" are the family's amount concepts minus its balance
concept (for
        banks: debit, credit). We try each amount column as the balance and assign the rest
to the
        flows, then also the no-per-row-balance case (total-reconcile families, e.g. Axis
Oct).
        """
        amount_cols = [c for c, k in sorted(col_kinds.items()) if k == "amount"]
        flows = [c for c in family.amount_concepts if c != family.balance_concept]
        bal = family.balance_concept
        cands: list[dict[str, int]] = []
        if bal and len(amount_cols) >= len(flows) + 1:
            for bcol in amount_cols:
                rest = [c for c in amount_cols if c != bcol]
                for assign in itertools.permutations(rest, len(flows)):
                    cands.append({**base, **dict(zip(flows, assign, strict=True)), bal:
bcol})
```

```
        if flows and len(amount_cols) >= len(flows):
            for assign in itertools.permutations(amount_cols, len(flows)):
                cands.append({**base, **dict(zip(flows, assign, strict=True))})
        return [("L2", c) for c in cands[:24]]


    def _apply_map(data: list[list[Cell]], col_map: dict[str, int]) -> list[dict[str, str]]:
        out: list[dict[str, str]] = []
        for row in data:
            out.append({c: (_text(row[i]) if i < len(row) else "") for c, i in
col_map.items()})
        return out


    def map_table(
        table: Sequence[Row] | None,  # Sequence (covariant) so list[list[str | None]]
callers fit
        profile: Profile,
        family: FamilySpec = BANK_FAMILY,
        *,
        oracle: Callable[[list], bool] | None = None,
    ) -> MappingResult | None:
        """Map one extracted table onto ``family``'s concepts; ``oracle`` selects among
candidates.

        Resolution order: header lexicon (L0) ? profile column order ? content search (L2).
With an
        ``oracle`` the first candidate it endorses wins (``verified=True``); without one,
the first
        structurally-complete candidate is returned unverified (the caller reconciles
separately).
        Returns ``None`` if no transaction rows are found, or the best unverified candidate
if an oracle
        was supplied but nothing reconciled.
        """
        if not table:
            return None
        rows_in = [list(r) for r in table if r is not None]
        if not rows_in:
            return None
        alias_index = _alias_index(profile, family)
        patterns = _date_patterns(profile)

        header_idx, col_map_hdr = _detect_header(rows_in, alias_index)
        start = header_idx + 1 if header_idx is not None else 0
        date_col_hint = col_map_hdr.get("txn_date")
        if date_col_hint is None and profile.column_order and "txn_date" in
profile.column_order:
            date_col_hint = profile.column_order.index("txn_date")

        data = [
            r
            for r in rows_in[start:]
            if not _is_balance_marker(r) and _is_transaction_row(r, date_col_hint, patterns)
        ]
        if not data:
            return None
        ncols = max(len(r) for r in data)
        col_kinds = _classify_columns(data, ncols, patterns)
        base = _base_map(col_kinds, data, family)
        flows = [c for c in family.amount_concepts if c != family.balance_concept]

        candidates: list[tuple[str, dict[str, int]]] = []
        if flows and all(c in col_map_hdr for c in flows):  # L0: header pinned the flow
concepts
```

```
            candidates.append(("L0", {**base, **col_map_hdr}))
        elif col_map_hdr:  # header present but didn't pin the flows: keep its hints, search
the rest
            candidates.append(("L0", {**base, **col_map_hdr}))
        if profile.column_order:  # positional prior (no/partial header)
            candidates.append(("profile-order", {c: i for i, c in
enumerate(profile.column_order)}))
        candidates += _search_candidates(base, col_kinds, family)

        primary_date = family.date_concepts[0] if family.date_concepts else None
        best: MappingResult | None = None
        for layer, cmap in candidates:
            if primary_date and primary_date not in cmap:
                continue
            if not (set(family.amount_concepts) & cmap.keys()):
                continue
            mapped = _apply_map(data, cmap)
            typed = family.build(mapped)
            if oracle is None:
                return MappingResult(typed, mapped, cmap, layer, verified=False)
            if oracle(typed):
                return MappingResult(typed, mapped, cmap, layer, verified=True)
            if best is None:
                best = MappingResult(typed, mapped, cmap, layer, verified=False)
        return best


    __all__ = ["Profile", "FamilySpec", "BANK_FAMILY", "MappingResult", "map_table"]

```

8. user (2026-06-10T17:59:39.220Z)

```
      """AU Small Finance Bank savings / transaction-account statement parser.

      Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
      coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
      vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
entirely in
      ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
here.
      """

      from __future__ import annotations

      import sqlite3
      from pathlib import Path

      from muneem import __version__
      from muneem import db as dbmod
      from muneem.errors import ParseError
      from muneem.redaction import redact_filename

      from . import register_parser
      from .base import StatementParser
      from .profiles import AUBANK_PROFILE
      from .savings import SavingsParse, parse_savings
      from .validation import Confidence


      def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
          """Parse an AU Bank savings statement via the shared engine-driven path."""
          return parse_savings(pdf_path, AUBANK_PROFILE, password)


```

```
    @register_parser
    class AUParser(StatementParser):
        def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
            if sender and "aubank" in sender.lower():
                return True
            low = (text or "").lower()
            # Text evidence must be letterhead/footer-shaped: the spelled-out bank name, or
the
            # bank's domain WITH its TLD ("aubank.in", as printed in statement footers). The
bare
            # token "aubank" also appears in UPI handles ("UPI/someone@aubank/?") inside
OTHER
            # banks' narrations ? and this parser is alphabetically first in the registry,
so a
            # bare substring match hijacked routing for any statement that merely paid an AU
user.
            return "au small finance bank" in low or "aubank.in" in low

        def parse_and_insert(
            self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
        ) -> int:
            """Persist a savings parse **only if it reconciles**; otherwise store
nothing."""
            try:
                parsed = parse_au_savings(pdf_path, password)
            except ParseError:
                raise
            except Exception as exc:
                raise ParseError(f"failed to parse AU statement
{redact_filename(pdf_path)}") from exc

            if not parsed.verified:
                dbmod.record_parse(
                    conn,
                    doc_id,
                    confidence=Confidence.UNRECONCILED.value,
                    parser="AUParser",
                    parser_version=__version__,
                )
                return 0

            for r in parsed.rows:
                dbmod.insert_au_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=parsed.account_number,
                    txn_date=r.txn_date,
                    value_date=r.value_date or "",
                    particulars=r.narration,
                    debit=r.debit,
                    credit=r.credit,
                    balance=r.balance,
                )
            dbmod.record_parse(
                conn,
                doc_id,
                confidence=Confidence.RECONCILED.value,
                parser="AUParser",
                parser_version=__version__,
            )
            return len(parsed.rows)

```

9. user (2026-06-10T17:59:39.522Z)

```
1       """Coordinate clustering ? turn positioned words into a clean cell-matrix.
2
3       When ``pdfplumber.extract_tables()`` mis-groups a semi-ruled or whitespace-aligned
statement (the
4       Axis case: a date merged into the narration cell, column counts drifting between
vintages), we read
5       **word positions** directly. Words are grouped into visual rows by their ``top``; column
boundaries
6       are inferred from the vertical whitespace **shared across the transaction rows**; and
each word is
7       dropped into the column its x-centre falls in. Because columns come from geometry, a
leading date
8       sits in its *own* column instead of being glued to the narration.
9
10      Two robustness lessons baked in:
11
12      * **Infer columns from transaction-like rows only.** Headers, address blocks, and
summaries span the
13        page and would erase the inter-column whitespace. A caller-supplied ``keep_line``
predicate (e.g.
14        "has a date *and* a money token") restricts both column inference and the emitted
matrix to the
15        transaction band.
16      * **Use a coverage histogram, not interval-merging.** A column boundary is an x-channel
empty
17        in *most* rows; a single long narration can't collapse two columns.
18
19      Bank-agnostic extraction (no issuer constants, no x-positions hard-coded); the engine's
oracle still
20      decides whether the result is trustworthy. Pure geometry in, strings out ? no value is
logged.
21      """
22
23      from __future__ import annotations
24
25      from collections.abc import Callable, Sequence
26
27      Word = dict  # a pdfplumber word: {"text", "x0", "x1", "top", "bottom", ...}
28
29
30      def group_lines(words: Sequence[Word], tol: float = 2.5) -> list[list[Word]]:
31          """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter), left-to-
right."""
32          lines: list[list[Word]] = []
33          anchor_top: float | None = None
34          for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
35              if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= tol:
36                  lines[-1].append(w)
37              else:
38                  lines.append([w])
39                  anchor_top = w["top"]
40          return [sorted(line, key=lambda w: w["x0"]) for line in lines]
41
42
43      def infer_columns(
44          lines: Sequence[Sequence[Word]], bin_w: float = 1.0, min_frac: float = 0.3
45      ) -> list[tuple[float, float]]:
46          """Infer column x-extents from a coverage histogram over ``lines``.
47
48          An x-bin is "filled" if at least ``min_frac`` of the lines have a word covering it;
columns are
49          the runs of filled bins, and the empty runs between them are the inter-column
whitespace. Robust
```

```
        to a few full-width lines: it thresholds on the *fraction* of rows, not on any single one.
        """
        words = [w for line in lines for w in line]
        if not words:
            return []
        x0 = min(float(w["x0"]) for w in words)
        x1 = max(float(w["x1"]) for w in words)
        nbins = max(1, int((x1 - x0) / bin_w) + 1)
        cover = [0] * nbins
        for line in lines:
            marked: set[int] = set()
            for w in line:
                a = int((float(w["x0"]) - x0) / bin_w)
                b = int((float(w["x1"]) - x0) / bin_w)
                marked.update(range(max(0, a), min(nbins - 1, b) + 1))
            for k in marked:
                cover[k] += 1
        thresh = max(1.0, min_frac * len(lines))
        filled = [c >= thresh for c in cover]
        cols: list[tuple[float, float]] = []
        start: int | None = None
        for i, f in enumerate(filled):
            if f and start is None:
                start = i
            elif not f and start is not None:
                cols.append((x0 + start * bin_w, x0 + i * bin_w))
                start = None
        if start is not None:
            cols.append((x0 + start * bin_w, x0 + nbins * bin_w))
        return cols


    def _column_of(center: float, columns: list[tuple[float, float]]) -> int:
        for i, (a, b) in enumerate(columns):
            if a <= center <= b:
                return i
        return min(
            range(len(columns)),
            key=lambda i: min(abs(center - columns[i][0]), abs(center - columns[i][1])),
        )


    def line_to_cells(line: Sequence[Word], columns: list[tuple[float, float]]) -> list[str]:
        """Place each word into its column; join words that share a column (in x order)."""
        cells = ["" for _ in columns]
        for w in line:
            idx = _column_of((float(w["x0"]) + float(w["x1"])) / 2, columns)
            cells[idx] = (cells[idx] + " " + str(w["text"])).strip()
        return cells


    def cluster_to_matrix(
        words_by_page: Sequence[Sequence[Word]],
        *,
        line_tol: float = 2.5,
        min_frac: float = 0.3,
        keep_line: Callable[[list[str]], bool] | None = None,
    ) -> list[list[str]]:
        """Cluster words across pages into one cell-matrix with consistent columns.

        ``keep_line`` (texts of a line ? bool) restricts both column inference and the output to the
        transaction band ? without it, full-width headers/footers swamp the column
```

```
structure. Columns
112         are inferred once across all kept lines (a statement keeps its layout); rows are
emitted in
113         page-then-top order.
114         """
115         lines: list[list[Word]] = []
116         for page in words_by_page:
117             lines.extend(group_lines(page, line_tol))
118         if keep_line is not None:
119             lines = [ln for ln in lines if keep_line([str(w["text"]) for w in ln])]
120         if not lines:
121             return []
122         columns = infer_columns(lines, min_frac=min_frac)
123         if not columns:
124             return []
125         return [line_to_cells(line, columns) for line in lines]
126
127
128     __all__ = ["group_lines", "infer_columns", "line_to_cells", "cluster_to_matrix"]
129
```

10. user (2026-06-10T17:59:44.556Z)

```
1       """The ``muneem`` command-line interface.
2
3       Track A ships the promo spine: ``ingest`` a PDF into the local store and
4       ``offers list`` to read back what was found. ``--ocr`` adds local image OCR for
5       codes rasterized into banners (approximate; flagged for verification). Pipeline
6       subcommands (``fetch``, ``txns``) land with their milestones.
7       """
8
9       from __future__ import annotations
10
11      from contextlib import contextmanager
12      from pathlib import Path
13
14      import typer
15
16      from muneem import __version__
17      from muneem import db as dbmod
18      from muneem.config import load_config
19      from muneem.log import get_logger
20      from muneem.redaction import redact_filename
21
22      _log = get_logger(__name__)
23
24      app = typer.Typer(
25          add_completion=False,
26          help="muneem ? a local-first personal-finance bookkeeper.",
27          no_args_is_help=True,
28      )
29      offers_app = typer.Typer(no_args_is_help=True, help="Work with extracted offers.")
30      app.add_typer(offers_app, name="offers")
31
32      txns_app = typer.Typer(no_args_is_help=True, help="Work with parsed transactions.")
33      app.add_typer(txns_app, name="txns")
34
35      db_app = typer.Typer(no_args_is_help=True, help="Manage the local encrypted ledger.")
36      app.add_typer(db_app, name="db")
37
38
39      @contextmanager
40      def _ledger(db_path: Path):
41          """Open the encrypted ledger, turning an un-migrated *plaintext* file into a clean
CLI
```

```
        error (pointing at ``muneem db encrypt``) rather than a raw traceback."""
        try:
            with dbmod.connect_encrypted(db_path) as conn:
                yield conn
        except dbmod.EncryptedDatabaseError as exc:
            typer.echo(str(exc), err=True)
            raise typer.Exit(1) from exc


    # Dispatch table for supported banks (extensible as new parsers land).
    BANK_DISPATCH = {
        "axis": (dbmod.list_axis_transactions, "debit", "credit"),
        "axis_cc": (dbmod.list_axis_cc_transactions, "amount", "amount"),  # direction from
"type"
        "au": (dbmod.list_au_transactions, "debit", "credit"),
        "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
    }


    def _version_callback(value: bool) -> None:
        if value:
            typer.echo(f"muneem {__version__}")
            raise typer.Exit()


    @app.callback()
    def main(
        version: bool = typer.Option(
            False,
            "--version",
            callback=_version_callback,
            is_eager=True,
            help="Show the muneem version and exit.",
        ),
    ) -> None:
        """muneem command-line interface."""


    @app.command()
    def info() -> None:
        """Show resolved local paths (no secrets, no sensitive data)."""
        cfg = load_config()
        present = "present" if cfg.config_file.is_file() else "absent"
        typer.echo(f"muneem {__version__}")
        typer.echo(f"config dir : {cfg.config_dir}")
        typer.echo(f"data dir   : {cfg.data_dir}")
        typer.echo(f"db path    : {cfg.db_path}")
        typer.echo(f"staging    : {cfg.staging_dir}")
        typer.echo(f"config file: {cfg.config_file} ({present})")


    def _ingest_bank_statement(pdf: Path, db_path: Path, doc_type: str, *, interactive:
bool) -> None:
        """Unlock (if encrypted) ? extract ? select parser ? store reconciled transactions.

        The hybrid password unlock (cached/configured candidates ? prompt) runs *before*
extraction,
        since the text needed to pick a parser is itself encrypted. The resolved password is
forwarded
        to extraction and the parser, used in-memory only ? never logged.
        """
        from muneem import db as dbmod
        from muneem.extract import extract
        from muneem.parsers import get_parser
        from muneem.statement_unlock import resolve_password
```

```
        from muneem.unlock import is_encrypted

        password: str | None = ""
        if is_encrypted(pdf):
            password = resolve_password(pdf, interactive=interactive)
            if password is None:
                hint = " (run without --no-prompt to enter it)" if not interactive else ""
                typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.",
err=True)
                raise typer.Exit(1)

        ex = extract(pdf, password=password or "")  # resolved above; `or ""` keeps the type
str
        parser = get_parser(ex.text)
        if parser is None:
            typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.",
err=True)
            raise typer.Exit(1)

        with _ledger(db_path) as conn:
            dbmod.init_db(conn)
            existing = dbmod.get_document_by_sha(conn, ex.sha256)
            if existing is not None and existing["status"] == "reconciled":
                typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
                return
            if existing is not None:
                # sha256 dedup must not entomb a failure: a document the oracle did NOT
endorse
                # gets a fresh parse (a parser fix deserves a second chance). Same document
id,
                # clean row slate; provenance upserts on the re-parse.
                doc_id = int(existing["id"])
                dbmod.clear_document_rows(conn, doc_id)
                typer.echo(
                    f"re-parsing previously {existing['status']} document (doc {doc_id}):
{pdf.name}"
                )
            else:
                doc_id = dbmod.insert_document(
                    conn,
                    source="file",
                    external_id=pdf.name,
                    doc_type=doc_type,
                    sha256=ex.sha256,
                    path=str(pdf.resolve()),
                    status="parsed",
                )
            count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
            doc = dbmod.get_document_by_sha(conn, ex.sha256)
            status = doc["status"] if doc else "parsed"

        typer.echo(
            f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
            f"via {parser.__class__.__name__} [{status}]"
        )


    def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
        """Run one statement through the full parse pipeline into an in-memory DB.

        Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document
status
        (``reconciled`` / ``unreconciled``) or a failure tag (``locked`` / ``no-parser`` /
``error``).
        Nothing is persisted and no statement value is surfaced ? only the verdict and
```

```
counts.
        """
        from muneem import db as dbmod
        from muneem.extract import extract
        from muneem.parsers import get_parser
        from muneem.statement_unlock import resolve_password
        from muneem.unlock import is_encrypted

        password = ""
        if is_encrypted(pdf):
            password = resolve_password(pdf, interactive=interactive) or ""
            if not password:
                return ("locked", 0, None)
        try:
            ex = extract(pdf, password=password)
            parser = get_parser(ex.text)
        except Exception as exc:  # eval must not crash on one bad file; record the redacted
reason
            _log.warning(
                "eval: extract/route failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
            )
            return ("error", 0, None)
        if parser is None:
            return ("no-parser", 0, None)
        name = parser.__class__.__name__
        try:
            with dbmod.connect(":memory:") as conn:
                dbmod.init_db(conn)
                doc_id = dbmod.insert_document(
                    conn, source="file", external_id=pdf.name, sha256=ex.sha256,
status="parsed"
                )
                count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
                doc = dbmod.get_document_by_sha(conn, ex.sha256)
                status = doc["status"] if doc else "parsed"
        except Exception as exc:
            _log.warning(
                "eval: parse/store failed for %s (%s)", redact_filename(pdf),
type(exc).__name__
            )
            return ("error", 0, name)
        return (status, count, name)


    @app.command("eval")
    def eval_statements(
        folder: Path = typer.Argument(
            ...,
            exists=True,
            file_okay=False,
            readable=True,
            help="A LOCAL folder of statement PDFs to evaluate (e.g. your seed or golden
set).",
        ),
        prompt: bool = typer.Option(
            False,
            "--prompt/--no-prompt",
            help="Prompt for a password when cached/configured candidates don't unlock a
file.",
        ),
    ) -> None:
        """Evaluate the parsers against a folder of statements and report the reconcile
rate.

```

```
        The seed?golden onboarding loop (see docs/onboarding-a-bank.md): point it at a few
*seed*
        statements while tuning a profile, then at a *golden* set to confirm. Everything
runs locally in
        an in-memory DB; only counts / statuses / parser names print ? never statement
values.
        """
        pdfs = sorted(folder.glob("*.pdf"))
        if not pdfs:
            typer.echo(f"no .pdf files in {folder}")
            return
        reconciled = 0
        for pdf in pdfs:
            outcome, count, name = _eval_statement(pdf, interactive=prompt)
            reconciled += outcome == "reconciled"
            typer.echo(f"  {outcome:12} {count:4d} row(s)  {name or '?':20} {pdf.name}")
        typer.echo(f"\n{reconciled}/{len(pdfs)} reconciled")


    @app.command()
    def ingest(
        pdf: Path = typer.Argument(
            ..., exists=True, dir_okay=False, readable=True, help="Path to a PDF to ingest."
        ),
        doc_type: str = typer.Option("promo", help="Document category."),
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
        ocr: bool = typer.Option(
            False, "--ocr", help="Also OCR images locally (Tesseract) for codes in banners."
        ),
        no_prompt: bool = typer.Option(
            False,
            "--no-prompt",
            help="Never prompt for a statement password; use only cached/configured
candidates.",
        ),
    ) -> None:
        """Store a PDF's offers (promo) or transactions (bank_statement) in the local DB."""
        from muneem.extract import extract
        from muneem.offers import extract_offer

        db_path = db or load_config().db_path

        if doc_type == "bank_statement":
            _ingest_bank_statement(pdf, db_path, doc_type, interactive=not no_prompt)
            return

        from muneem import db as dbmod

        ex = extract(pdf)
        text_fields = extract_offer(ex.text)
        # (code, source) pairs; text-layer codes are exact, OCR codes are approximate.
        found: list[tuple[str, str]] = [(c, "text") for c in text_fields.codes]
        expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type

        if ocr:
            from muneem.ocr import ocr_pdf, tesseract_available

            if not tesseract_available():
                typer.echo("warning: --ocr requested but Tesseract not found; skipping
OCR.", err=True)
            else:
                ocr_fields = extract_offer(ocr_pdf(pdf))
                known = {c for c, _ in found}
                for code in ocr_fields.codes:
```

```
                    if code not in known:
                        found.append((code, "ocr"))
                        known.add(code)
                if expiry_raw is None:
                    expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type

        with _ledger(db_path) as conn:
            dbmod.init_db(conn)
            existing = dbmod.get_document_by_sha(conn, ex.sha256)
            if existing is not None:
                typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
                return
            doc_id = dbmod.insert_document(
                conn,
                source="file",
                external_id=pdf.name,
                doc_type=doc_type,
                sha256=ex.sha256,
                path=str(pdf.resolve()),
                status="parsed",
            )
            for code, src in found:
                dbmod.insert_offer(
                    conn,
                    document_id=doc_id,
                    promo_code=code,
                    expiry_raw=expiry_raw,
                    expiry_type=expiry_type,
                    source=src,
                    source_text=ex.text[:2000],
                )

        summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
        typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")


    @offers_app.command("list")
    def offers_list(
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
    ) -> None:
        """List extracted offers."""
        from muneem import db as dbmod

        db_path = db or load_config().db_path
        if not Path(db_path).exists():
            typer.echo("no offers yet (database not created).")
            return

        with _ledger(db_path) as conn:
            dbmod.init_db(conn)
            rows = dbmod.list_offers(conn)

        if not rows:
            typer.echo("no offers found.")
            return
        for row in rows:
            who = row["merchant"] or row["external_id"] or "?"
            when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
            tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
            typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")


    @txns_app.command("list")
    def txns_list(
```

```
        bank: str = typer.Argument(
            ..., help="Which bank to list (e.g. hdfc, axis) ? or 'all' for the cross-bank
view."
        ),
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
        limit: int = typer.Option(50, help="Max number of transactions to show."),
    ) -> None:
        """List parsed transactions for one bank, or across all banks via the serving
view."""
        from muneem import db as dbmod

        db_path = db or load_config().db_path
        if not Path(db_path).exists():
            typer.echo("database not created yet.")
            return

        bank = bank.lower()
        if bank == "all":
            with _ledger(db_path) as conn:
                dbmod.init_db(conn)
                rows = dbmod.list_all_transactions(conn, limit=limit)
            if not rows:
                typer.echo("no transactions found.")
                return
            for row in rows:
                amount = f"+{row['credit']}" if row["credit"] else f"-{row['debit']}"
                part = (row["particulars"] or "")[:40]
                verdict = row["confidence"] or "?"
                typer.echo(
                    f"[{row['bank']}:{row['account_number']}] {row['txn_date']} : "
                    f"{part:<40} | {amount:>10} [{verdict}]"
                )
            return

        if bank not in BANK_DISPATCH:
            typer.echo(f"Unknown bank: {bank}", err=True)
            return

        list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
        with _ledger(db_path) as conn:
            dbmod.init_db(conn)
            rows = list_fn(conn, limit=limit)

        if not rows:
            typer.echo(f"no {bank} transactions found.")
            return
        for row in rows:
            if bank == "axis_cc":
                sign = "+" if row["type"] == "Cr" else "-"
                amount = f"{sign}{row['amount']}"
            else:
                amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
            part = (row["particulars"] or "")[:50]
            # Anything but a clean 'reconciled' verdict is surfaced, never silently listed.
            status = row["doc_status"]
            tag = "" if status == "reconciled" else f" [{status}]"
            typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}{tag}")


    @db_app.command("status")
    def db_status(
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
```

```
    ) -> None:
        """Show the ledger's encryption state (no statement data is read or printed)."""
        from muneem import db as dbmod
        from muneem.db_key import peek_key, peek_pending_key

        db_path = db or load_config().db_path
        key_state = "present in keychain" if peek_key() else "not yet minted"
        if not Path(db_path).exists():
            typer.echo(f"ledger : {db_path} (does not exist yet)")
            typer.echo(f"key    : {key_state}")
            return
        if dbmod.is_plaintext_sqlite(db_path):
            state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
        else:
            state = "encrypted (SQLCipher)"
        typer.echo(f"ledger : {db_path}")
        typer.echo(f"state  : {state}")
        typer.echo(f"key    : {key_state}")
        if peek_pending_key():
            typer.echo(
                "pending: a staged rotation key is present (an interrupted 'db rotate-key'?)
? "
                "the next ledger open reconciles it automatically."
            )


    @db_app.command("encrypt")
    def db_encrypt(
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
    ) -> None:
        """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
place."""
        from muneem import db as dbmod
        from muneem.db_key import get_or_create_key

        db_path = db or load_config().db_path
        if not Path(db_path).exists():
            typer.echo(
                f"nothing to encrypt: {db_path} does not exist (a new ledger is born
encrypted)."
            )
            return
        if not dbmod.is_plaintext_sqlite(db_path):
            typer.echo(
                f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
to do."
            )
            return
        n = dbmod.encrypt_database(db_path, get_or_create_key())
        typer.echo(
            f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
remains at "
            f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
        )


    def _require_encrypted_ledger(db_path: Path) -> str:
        """Return the ledger's keychain key, or exit with a clear message if it can't be
used."""
        from muneem import db as dbmod
        from muneem.db_key import peek_key

        if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
            typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
```

```
err=True)
            raise typer.Exit(1)
        key = peek_key()
        if not key:
            typer.echo("no encryption key in the keychain for this ledger.", err=True)
            raise typer.Exit(1)
        return key


    @db_app.command("rotate-key")
    def db_rotate_key(
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
    ) -> None:
        """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
        from muneem import db as dbmod
        from muneem import db_key
        from muneem.errors import KeychainError

        db_path = db or load_config().db_path
        current = _require_encrypted_ledger(db_path)
        fresh = db_key.new_key()
        # Both keys keep a durable copy at every instant: stage the fresh key under the
PENDING
        # keychain entry, rekey the file, and only then promote it to the primary entry. Any
        # interruption leaves a state that connect_encrypted can heal (file rekeyed ->
pending key
        # promoted on next open; file untouched -> primary entry still matches it).
        db_key.set_pending_key(fresh)
        try:
            dbmod.rekey_database(db_path, current, fresh)
        except Exception as exc:
            # Don't trust the exception to tell us how far the rekey got ? check the file.
            if dbmod.key_opens_database(db_path, fresh):
                pass  # the rekey actually completed; fall through to promotion below
            elif dbmod.key_opens_database(db_path, current):
                db_key.clear_pending_key()
                typer.echo(
                    f"key rotation failed; the ledger is unchanged (still under the existing
key): "
                    f"{exc}",
                    err=True,
                )
                raise typer.Exit(1) from exc
            else:
                typer.echo(
                    "key rotation failed and the ledger opens with neither key. Both keys
were kept "
                    "in the keychain ('db-key' = old, 'db-key-pending' = new); restore the
ledger "
                    "from a backup. Do NOT delete either keychain entry until the ledger
opens.",
                    err=True,
                )
                raise typer.Exit(1) from exc
        try:
            db_key.set_key(fresh)  # promote: the file is verifiably under the fresh key now
        except KeychainError as exc:
            typer.echo(
                "the ledger was rekeyed but promoting the new key in the keychain failed;
the new "
                "key is safe under 'db-key-pending' and the next ledger open will finish the
"
                "promotion automatically.",
                err=True,
```

```
            )
            raise typer.Exit(1) from exc
        db_key.clear_pending_key()
        typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
        typer.echo(
            "note: backups and any recovery file created BEFORE this rotation still use the
old "
            "key. Re-run 'muneem db backup' and 'muneem db setup-recovery' now."
        )


    @db_app.command("backup")
    def db_backup(
        dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
copy."),
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
    ) -> None:
        """Write an encrypted online-backup copy of the ledger (consistent even with WAL
active)."""
        from muneem import db as dbmod

        db_path = db or load_config().db_path
        key = _require_encrypted_ledger(db_path)
        try:
            dbmod.backup_database(db_path, dest, key)
        except dbmod.EncryptedDatabaseError as exc:
            typer.echo(str(exc), err=True)
            raise typer.Exit(1) from exc
        typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")


    @db_app.command("setup-recovery")
    def db_setup_recovery(
        out: Path | None = typer.Option(
            None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
        ),
    ) -> None:
        """Create a passphrase-wrapped recovery file for the ledger's encryption key.

        Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
lost. The
        file holds only the *wrapped* key ? useless without the passphrase.
        """
        import getpass

        from muneem import recovery
        from muneem.db_key import peek_key

        key = peek_key()
        if not key:
            typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
            raise typer.Exit(1)
        out_path = out or (load_config().data_dir / "muneem-recovery.json")
        passphrase = getpass.getpass("New recovery passphrase: ")
        if passphrase != getpass.getpass("Confirm passphrase: "):
            typer.echo("passphrases did not match.", err=True)
            raise typer.Exit(1)
        try:
            recovery.write_recovery_file(out_path, key, passphrase)
        except recovery.RecoveryError as exc:
            typer.echo(str(exc), err=True)
            raise typer.Exit(1) from exc
        typer.echo(
```

```
            f"wrote recovery file to {out_path}. Store it OFF this machine (password manager / "
            "separate drive) and remember the passphrase ? you need BOTH to recover. Without them, a "
            "lost keychain means the ledger and its backups are unrecoverable."
        )


    @db_app.command("recover")
    def db_recover(
        file: Path = typer.Argument(..., exists=True, dir_okay=False, help="Recovery file path."),
        db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data dir)."),
    ) -> None:
        """Restore the ledger's encryption key from a recovery file + passphrase (into the keychain)."""
        import getpass

        from muneem import db as dbmod
        from muneem import db_key, recovery

        passphrase = getpass.getpass("Recovery passphrase: ")
        try:
            key = recovery.read_recovery_file(file, passphrase)
        except recovery.RecoveryError as exc:
            typer.echo(str(exc), err=True)
            raise typer.Exit(1) from exc
        db_path = db or load_config().db_path
        # If the ledger is present, verify the recovered key actually opens it before committing it.
        if Path(db_path).exists() and not dbmod.is_plaintext_sqlite(db_path):
            try:
                with dbmod.connect(db_path, key=key) as conn:
                    conn.execute("SELECT 1")
            except dbmod.EncryptedDatabaseError as exc:
                typer.echo("recovered a key, but it does not open the current ledger.", err=True)
                raise typer.Exit(1) from exc
        db_key.set_key(key)
        typer.echo("recovered the encryption key into the keychain.")


    if __name__ == "__main__":
        app()

```

11. user (2026-06-10T17:59:50.588Z)

```
      """Resolve a statement PDF's password for the ingest path ? the hybrid strategy, wired.

      DESIGN.md §8.2 / ROADMAP C.1. Resolution order when a statement is encrypted:

      1. **Candidate dictionary** ? passwords cached in the OS keychain (from earlier successful
         unlocks) plus those configured in ``password_rules.yaml`` (see
         :mod:`muneem.password_rules`).
      2. **Interactive prompt** ? a hidden ``getpass`` prompt (skipped with ``interactive=False``,
         e.g. automation / CI).

      A password that works **via the prompt** is cached in the keychain so the next ingest is
      friction-free. Nothing here logs a password, writes one to the repo, or echoes it; the cache is
      the OS keychain (:mod:`muneem.secrets`), never a file.
```

```
    """

    from __future__ import annotations

    import getpass
    import json
    from pathlib import Path

    from muneem import secrets
    from muneem.errors import KeychainError
    from muneem.password_rules import candidate_passwords
    from muneem.unlock import is_encrypted, unlock

    # Keychain entry holding a JSON list of known-good statement passwords (the learned
candidates).
    _CACHE_KEY = "statement_unlock_candidates"
    # Bound the cached list so the keychain entry can't grow without limit; most-recently-
used kept.
    _MAX_CACHED = 20


    def _cached_candidates() -> list[str]:
        try:
            raw = secrets.get_secret(_CACHE_KEY)
        except KeychainError:
            return []  # an unavailable keychain just means no cache this run
        if not raw:
            return []
        try:
            return [p for p in json.loads(raw) if isinstance(p, str) and p]
        except (ValueError, TypeError):
            return []


    def _remember(password: str) -> None:
        """Cache a working password in the keychain (MRU order, deduped, capped). Never
logged.

        Best-effort: a keychain write failure is swallowed (the password already unlocked
the file this
        run); we never crash an ingest over a caching hiccup.
        """
        if not password or not password.strip():
            return
        current = [p for p in _cached_candidates() if p != password]
        updated = [password, *current][:_MAX_CACHED]  # most-recently-used first, bounded
        try:
            secrets.set_secret(_CACHE_KEY, json.dumps(updated))
        except KeychainError:
            pass


    def resolve_password(pdf_path: Path | str, *, interactive: bool = True) -> str | None:
        """Return a working password for ``pdf_path``.

        ``""`` means the PDF is not encrypted (no password needed). ``None`` means it is
encrypted and
        no candidate (or prompt) unlocked it. A prompt-resolved password is cached for next
time.
        """
        path = Path(pdf_path)
        if not is_encrypted(path):
            return ""

        candidates = [*_cached_candidates(), *candidate_passwords()]
```

```
        prompt_fn = None
        if interactive:

            def prompt_fn() -> str | None:  # noqa: E306 ? local prompt; no echo, never
logged
                return getpass.getpass(f"Password for {path.name}: ") or None

        result = unlock(path, candidates=candidates, prompt_fn=prompt_fn)
        if not result.ok:
            return None
        if result.method == "prompt" and result.password:
            _remember(result.password)
        return result.password or ""

```

12. user (2026-06-10T17:59:50.961Z)

```
    """Password rule loader for statement decryption (hybrid strategy support).

    Prefers the user config dir (via platformdirs, e.g. ~/.config/muneem/password_rules.yaml
    on Linux) for real rules. Falls back to legacy CWD "password_rules.yaml" (common during
    dev) with a DeprecationWarning. **Never commit real rules or the file itself** ?
`.gitignore`
    blocks `password_rules.*`.

    See DESIGN.md §8.2, CLAUDE.md security rules, and docs/ingesting-statements.md for the
file
    format. Entries are either ``<bank>: <password>`` or a top-level ``candidates: [..]``
list.
    """

    from __future__ import annotations

    import warnings
    from pathlib import Path
    from typing import Any

    import yaml

    from muneem.config import CONFIG_DIR
    from muneem.errors import ConfigError


    def _load_rules() -> dict[str, Any]:
        """Load the rules dict from the config dir (preferred) or legacy CWD file; ``{}`` if
absent.

        Resolution order:
        1. ``<config_dir>/password_rules.yaml``  (recommended location)
        2. ``./password_rules.yaml``  (legacy, emits DeprecationWarning)
        """
        for path, is_legacy in (
            (CONFIG_DIR / "password_rules.yaml", False),
            (Path("password_rules.yaml"), True),
        ):
            if path.exists():
                if is_legacy:
                    warnings.warn(
                        "Loading password_rules from CWD (./password_rules.yaml) is
deprecated. "
                        f"Move it to {CONFIG_DIR / 'password_rules.yaml'} for future-
proofing. "
                        "See docs and CLAUDE.md (never commit real password material).",
                        DeprecationWarning,
                        stacklevel=3,
```

```
                    )
                with path.open(encoding="utf-8") as f:
                    try:
                        data = yaml.safe_load(f)
                    except yaml.YAMLError as exc:
                        # Never echo the file's contents in the error ? they are password
material.
                        raise ConfigError(
                            f"could not parse password rules ({path.name}): invalid YAML"
                        ) from exc
                if data is None:
                    return {}
                if not isinstance(data, dict):
                    raise ConfigError(
                        f"password rules ({path.name}) must be a mapping of <bank>:
<password>"
                    )
                return data
        return {}


    def get_password_for_bank(bank_name: str) -> str | None:
        """Return the password (or derived rule string) for a bank, or ``None``."""
        val = _load_rules().get(bank_name)
        return str(val) if val else None


    def candidate_passwords() -> list[str]:
        """Every configured password to try as an unlock candidate, **bank-agnostic**.

        The ingest path resolves the password *before* it knows the issuer (it needs the
decrypted
        text to pick a parser), so it tries all configured passwords. Collects each per-bank
value
        plus any entries under a top-level ``candidates:`` list; order preserved, de-
duplicated.
        Quote numeric passwords in the YAML so leading zeros survive.
        """
        rules = _load_rules()
        collected: list[str] = []
        for key, val in rules.items():
            if key == "candidates" and isinstance(val, list):
                collected.extend(str(v) for v in val if v not in (None, ""))
            elif isinstance(val, str | int | float):
                collected.append(str(val))
        seen: set[str] = set()
        unique: list[str] = []
        for pw in collected:
            if pw and pw not in seen:
                seen.add(pw)
                unique.append(pw)
        return unique

```

13. user (2026-06-10T17:59:56.010Z)

```
    import pytest


    @pytest.fixture(autouse=True)
    def _isolate_keychain(monkeypatch):
        """Never touch the real OS keychain in tests.

        Replaces :mod:`muneem.secrets` with an in-memory store so the suite cannot pollute
the
```

```
9           developer's keychain (e.g. minting a DB key) and runs cleanly in headless CI where
no
10          keyring backend exists. Tests that need specific keychain behaviour re-patch on top.
11          """
12          store: dict[str, str] = {}
13
14          def _set(name, value):
15              store[name] = value
16
17          monkeypatch.setattr("muneem.secrets.get_secret", store.get)
18          monkeypatch.setattr("muneem.secrets.set_secret", _set)
19          monkeypatch.setattr("muneem.secrets.delete_secret", lambda name: store.pop(name,
None))
20          return store
21
22
23      # NOTE: the old `snapshot` fixture (testdata/snapshots/*.json) was removed deliberately.
24      # It wrote REAL parsed transaction rows to cleartext JSON inside the repo working dir ?
25      # the very data the encrypted ledger exists to protect ? and no test used it. If parse
26      # snapshotting returns, it must store only non-sensitive summaries (counts, reconcile
27      # booleans, hashes), never raw rows.
28
```

14. user (2026-06-10T17:59:56.620Z)

```
1       from pathlib import Path
2
3       from typer.testing import CliRunner
4
5       from muneem.cli import app
6       from muneem.extract import Extracted
7
8       runner = CliRunner()
9
10
11      def test_version_flag():
12          result = runner.invoke(app, ["--version"])
13          assert result.exit_code == 0
14          assert "muneem" in result.stdout
15
16
17      def test_info_lists_paths():
18          result = runner.invoke(app, ["info"])
19          assert result.exit_code == 0
20          assert "config dir" in result.stdout
21          assert "staging" in result.stdout
22
23
24      def test_ingest_bank_statement_unlocks_and_passes_password(tmp_path, monkeypatch):
25          """The bank path resolves the password and forwards it to extract + the parser."""
26          pdf = tmp_path / "stmt.pdf"
27          pdf.write_bytes(b"%PDF-1.4 fake")
28          captured = {}
29
30          monkeypatch.setattr("muneem.unlock.is_encrypted", lambda p: True)
31          monkeypatch.setattr(
32              "muneem.statement_unlock.resolve_password", lambda p, interactive=True: "PW123"
33          )
34
35          def fake_extract(p, password=""):
36              captured["extract_pw"] = password
37              return Extracted(path=Path(p), sha256="sha-x", text="Axis Bank Account
Statement")
38
39          monkeypatch.setattr("muneem.extract.extract", fake_extract)
```

```
40
41          class FakeParser:
42              def parse_and_insert(self, pdf_path, conn, doc_id, password=None):
43                  captured["parser_pw"] = password
44                  return 3
45
46          monkeypatch.setattr("muneem.parsers.get_parser", lambda text: FakeParser())
47
48          result = runner.invoke(
49              app, ["ingest", str(pdf), "--doc-type", "bank_statement", "--db", str(tmp_path /
"t.db")]
50          )
51          assert result.exit_code == 0, result.output
52          assert captured["extract_pw"] == "PW123" and captured["parser_pw"] == "PW123"
53          assert "ingested" in result.output and "FakeParser" in result.output
54
55
56      def test_quarantined_document_can_be_reingested_after_parser_fix(tmp_path, monkeypatch):
57          """sha256 dedup must not entomb a failed parse: unreconciled docs get a second
chance,
58          reconciled docs stay deduped."""
59          from muneem import db as dbmod
60          from muneem.parsers.validation import Confidence
61
62          pdf = tmp_path / "stmt.pdf"
63          pdf.write_bytes(b"%PDF-1.4 fake")
64          db = tmp_path / "t.db"
65          monkeypatch.setattr("muneem.unlock.is_encrypted", lambda p: False)
66          monkeypatch.setattr(
67              "muneem.extract.extract",
68              lambda p, password="": Extracted(
69                  path=Path(p), sha256="sha-q", text="Axis Bank Account Statement"
70              ),
71          )
72
73          class BrokenParser:  # the pre-fix parser: quarantines, stores nothing
74              def parse_and_insert(self, pdf_path, conn, doc_id, password=None):
75                  dbmod.record_parse(
76                      conn, doc_id, confidence=Confidence.UNRECONCILED.value,
parser="BrokenParser"
77                  )
78                  return 0
79
80          class FixedParser:  # after the parser fix: stores a row and reconciles
81              def parse_and_insert(self, pdf_path, conn, doc_id, password=None):
82                  dbmod.insert_axis_transaction(
83                      conn,
84                      document_id=doc_id,
85                      account_number="AX1",
86                      txn_date="01/01/2026",
87                      value_date="01/01/2026",
88                      particulars="recovered row",
89                      debit=100.0,
90                      credit=None,
91                      balance=900.0,
92                  )
93                  dbmod.record_parse(
94                      conn, doc_id, confidence=Confidence.RECONCILED.value,
parser="FixedParser"
95                  )
96                  return 1
97
98          args = ["ingest", str(pdf), "--doc-type", "bank_statement", "--db", str(db)]
99
100         monkeypatch.setattr("muneem.parsers.get_parser", lambda text: BrokenParser())
```

```
        first = runner.invoke(app, args)
        assert first.exit_code == 0, first.output
        assert "[unreconciled]" in first.output

        monkeypatch.setattr("muneem.parsers.get_parser", lambda text: FixedParser())
        second = runner.invoke(app, args)
        assert second.exit_code == 0, second.output
        assert "re-parsing previously unreconciled" in second.output
        assert "[reconciled]" in second.output

        third = runner.invoke(app, args)  # now trusted -> dedup holds
        assert third.exit_code == 0, third.output
        assert "already ingested" in third.output

        with dbmod.connect_encrypted(db) as conn:
            doc = dbmod.get_document_by_sha(conn, "sha-q")
            assert doc["status"] == "reconciled"
            rows = dbmod.list_axis_transactions(conn, limit=10)
            assert len(rows) == 1  # the re-parse started from a clean slate (no duplicates)


    def test_db_status_on_missing_ledger(tmp_path):
        result = runner.invoke(app, ["db", "status", "--db", str(tmp_path / "nope.db")])
        assert result.exit_code == 0, result.output
        assert "does not exist yet" in result.output
        assert "not yet minted" in result.output


    def test_db_encrypt_migrates_and_status_reflects_it(tmp_path):
        import sqlite3

        p = tmp_path / "ledger.db"
        con = sqlite3.connect(p)
        con.execute("CREATE TABLE t (id INTEGER PRIMARY KEY, amount REAL)")
        con.execute("INSERT INTO t (amount) VALUES (10.0)")
        con.commit()
        con.close()

        before = runner.invoke(app, ["db", "status", "--db", str(p)])
        assert before.exit_code == 0 and "PLAINTEXT" in before.output

        enc = runner.invoke(app, ["db", "encrypt", "--db", str(p)])
        assert enc.exit_code == 0, enc.output
        assert "migrated" in enc.output and "plaintext.bak" in enc.output

        after = runner.invoke(app, ["db", "status", "--db", str(p)])
        assert after.exit_code == 0 and "encrypted" in after.output


    def test_ingest_bank_statement_unlock_failure_exits_nonzero(tmp_path, monkeypatch):
        pdf = tmp_path / "stmt.pdf"
        pdf.write_bytes(b"%PDF-1.4 fake")
        monkeypatch.setattr("muneem.unlock.is_encrypted", lambda p: True)
        monkeypatch.setattr(
            "muneem.statement_unlock.resolve_password", lambda p, interactive=True: None
        )
        result = runner.invoke(
            app,
            [
                "ingest",
                str(pdf),
                "--doc-type",
                "bank_statement",
                "--db",
                str(tmp_path / "t.db"),
```

```
                "--no-prompt",
            ],
        )
        assert result.exit_code == 1
        assert "Could not unlock" in result.output

```

15. user (2026-06-10T18:00:02.485Z)

```
    """HDFC savings / transaction-account statement parser.

    **Approach:** coordinate / column clustering on **pdfplumber (MIT)** ? not the
    default ``extract_tables()`` and not a ready-made library. HDFC's transaction
    table is *semi-ruled*, where ``extract_tables()`` is unreliable; the deep-research
    decision (see the ``hdfc-parser-tech-decision`` memory) is to cluster words by
    position. pdfplumber's whole chain is permissive and it opens password-protected
    statements directly.

    **Layout.** Each transaction is one *primary* line ? the transaction date at the
    far left (``x0?52``) plus three right-aligned amount columns (Withdrawal, Deposit,
    Closing Balance) ? followed by zero or more *continuation* lines that wrap the
    narration and carry the value date (``dd/mm/yyyy`` at ``x0>110``) and the
    cheque/reference number. We group words into visual lines by ``top``, detect
    primary lines by a leading date + a closing-balance amount, attach continuations,
    and read columns by x-position.

    **Correctness is checked, not assumed.** ``reconcile`` verifies the running
    balance (closing[i] == closing[i-1] - withdrawal + deposit) and
    ``validate_against_summary`` cross-checks the parse against HDFC's own end-of-
    statement SUMMARY (Dr/Cr counts + opening/closing balance). Together they prove a
    parse complete without inspecting any individual value ? which is how we validate
    against real (sensitive) statements. No statement content is logged here.
    """

    from __future__ import annotations

    import csv
    import re
    from dataclasses import dataclass, field
    from pathlib import Path

    import pdfplumber

    from muneem import __version__
    from muneem import db as dbmod
    from muneem.errors import ParseError
    from muneem.redaction import redact_filename

    from . import register_parser
    from .base import StatementParser
    from .concepts import ConceptRow, StatementConcepts
    from .dates import normalize_statement_date
    from .profiles import HDFC_PROFILE
    from .validation import (
        BALANCE_TOLERANCE,
        BookendReport,
        Confidence,
        ReconcileReport,
        reconcile_running_balance,
        verify,
    )

    # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) ---------
    # Left-aligned columns key on x0; right-aligned amount columns key on their right
    # edge (x1), which is stable regardless of the number's magnitude.
```

```
    _DATE_MAX_X0 = 70.0  # transaction date sits at x0?52; the value date is at x0>110
    _NARR_MIN_X0 = 100.0  # narration band starts at x0?111
    _AMOUNT_MIN_X0 = 295.0  # amount columns start at x0?306; narration never crosses ?290
    _WITHDRAWAL_MAX_X1 = 400.0  # withdrawal right edge ?358
    _DEPOSIT_MAX_X1 = 490.0  # deposit right edge ?443; closing balance right edge ?561
    _LINE_TOL = 2.5  # words within this many points of each other share a visual line
    _CONT_MAX_SPAN = 70.0  # a continuation line is at most this far below its primary line

    _DATE_RE = re.compile(r"^\d{2}/\d{2}/\d{4}$")
    # Indian-format money: optional minus, comma groups, exactly two decimals, optional
Cr/Dr.
    _AMOUNT_RE = re.compile(r"^-?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
    _FOOTER_MARKERS = {"STATEMENT", "SUMMARY"}


    @dataclass
    class Transaction:
        date: str  # dd/mm/yyyy, as printed
        narration: str
        withdrawal: float
        deposit: float
        closing_balance: float
        value_date: str | None = None
        ref: str | None = None
        page: int = 0

        def to_concept(self) -> ConceptRow:
            """Map HDFC's native row onto the bank-agnostic concept vocabulary (the
verifier's input).

            Withdrawal ? ``debit``, deposit ? ``credit``, closing balance ? ``balance``. The
``page``
            field is HDFC-internal provenance and has no concept equivalent.
            """
            return ConceptRow(
                txn_date=self.date,
                narration=self.narration,
                debit=self.withdrawal,
                credit=self.deposit,
                balance=self.closing_balance,
                value_date=self.value_date,
                reference=self.ref,
            )


    @dataclass
    class Statement:
        path: Path
        transactions: list[Transaction] = field(default_factory=list)
        account_number: str | None = None


    # HDFC's SUMMARY block and its cross-check are now expressed in the shared concept
vocabulary +
    # verifier (the concept-core refactor). These back-compat names keep HDFC's public
surface stable:
    #   StatementSummary  == the statement-level concepts a SUMMARY block supplies
    #   ValidationReport  == the verifier's bookend report (per-row walk + printed-figure
checks)
    StatementSummary = StatementConcepts
    ValidationReport = BookendReport


    def _to_amount(token: str) -> float | None:
        """Parse an HDFC money token to a float, or None if it is not money.
```

```
        Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not None): a zero amount
        is real information and the reconciliation math needs it.
        """
        if not _AMOUNT_RE.match(token):
            return None
        cleaned = re.sub(r"(?i)(cr|dr)$", "", token).replace(",", "")
        try:
            return float(cleaned)
        except ValueError:
            return None


    def _group_lines(words: list[dict]) -> list[list[dict]]:
        """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter)."""
        lines: list[list[dict]] = []
        anchor_top: float | None = None
        for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
            if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= _LINE_TOL:
                lines[-1].append(w)
            else:
                lines.append([w])
                anchor_top = w["top"]
        return [sorted(line, key=lambda w: w["x0"]) for line in lines]


    def _line_top(line: list[dict]) -> float:
        return min(w["top"] for w in line)


    def _has_narration(line: list[dict]) -> bool:
        return any(_NARR_MIN_X0 <= w["x0"] < _AMOUNT_MIN_X0 for w in line)


    def _has_amount(line: list[dict]) -> bool:
        return any(w["x0"] >= _AMOUNT_MIN_X0 and _AMOUNT_RE.match(w["text"]) for w in line)


    def _has_leading_date(line: list[dict]) -> bool:
        first = line[0]
        return bool(_DATE_RE.match(first["text"]) and first["x0"] < _DATE_MAX_X0)


    def _has_footer_marker(line: list[dict]) -> bool:
        return any(w["text"].upper() in _FOOTER_MARKERS for w in line)


    def _is_primary(line: list[dict]) -> bool:
        """A transaction's first line: a date at the far left + a closing-balance amount.

        Requiring the closing-balance amount guards against stray dates in the page
        header / account-summary box (e.g. the statement period).
        """
        if not _has_leading_date(line):
            return False
        return any(
            w["x0"] >= _AMOUNT_MIN_X0 and w["x1"] >= _DEPOSIT_MAX_X1 and
_AMOUNT_RE.match(w["text"])
            for w in line
        )


    def _is_continuation(line: list[dict]) -> bool:
        """A narration-wrap line: narration-band text, no leading date, no amounts."""
        return _has_narration(line) and not _has_leading_date(line) and not
```

```
_has_amount(line)


    def _parse_transaction(
        primary: list[dict], continuations: list[list[dict]], page_no: int
    ) -> Transaction:
        withdrawal = deposit = 0.0
        closing: float | None = None
        narr: list[tuple[float, float, str]] = []  # (top, x0, text) for ordered reassembly

        for line in (primary, *continuations):
            for w in line:
                x0, x1, text = w["x0"], w["x1"], w["text"]
                if x0 >= _AMOUNT_MIN_X0:
                    val = _to_amount(text)
                    if val is None:
                        continue
                    if x1 < _WITHDRAWAL_MAX_X1:
                        withdrawal = val
                    elif x1 < _DEPOSIT_MAX_X1:
                        deposit = val
                    else:
                        closing = val
                elif x0 >= _NARR_MIN_X0:
                    narr.append((w["top"], x0, text))

        value_date: str | None = None
        parts: list[str] = []
        for _top, _x0, text in sorted(narr):
            if value_date is None and _DATE_RE.match(text):
                value_date = text  # the value date, lifted out of the narration band
                continue
            parts.append(text)

        ref_candidates = [p for p in parts if p.isdigit() and len(p) >= 9]
        ref = max(ref_candidates, key=len) if ref_candidates else None

        return Transaction(
            date=primary[0]["text"],
            narration=" ".join(parts),
            withdrawal=withdrawal,
            deposit=deposit,
            closing_balance=closing if closing is not None else 0.0,
            value_date=value_date,
            ref=ref,
            page=page_no,
        )


    def extract_transactions_from_lines(lines: list[list[dict]], page_no: int = 0) ->
list[Transaction]:
        """Pure, PDF-free core: turn grouped table-body lines into transactions.

        Kept separate from PDF I/O so the column logic is unit-testable on synthetic
        word lists, with no real data.
        """
        primaries = [i for i, line in enumerate(lines) if _is_primary(line)]
        transactions: list[Transaction] = []
        for k, idx in enumerate(primaries):
            stop = primaries[k + 1] if k + 1 < len(primaries) else len(lines)
            primary = lines[idx]
            primary_top = _line_top(primary)
            continuations: list[list[dict]] = []
            for m in range(idx + 1, stop):
                line = lines[m]
```

```
                if _has_footer_marker(line) or _line_top(line) - primary_top >
_CONT_MAX_SPAN:
                    break
                if _is_continuation(line):
                    continuations.append(line)
            transactions.append(_parse_transaction(primary, continuations, page_no))
        return transactions


    def _table_body(lines: list[list[dict]]) -> list[list[dict]]:
        """Lines below the column header (drops the per-page account-summary box)."""
        header_top: float | None = None
        for line in lines:
            texts = {w["text"] for w in line}
            if "Narration" in texts and any(t.startswith("Closing") for t in texts):
                header_top = _line_top(line)
                break
        if header_top is None:
            return []
        return [line for line in lines if _line_top(line) > header_top + 1]


    def _extract_account_number(text: str) -> str | None:
        """Account number via the HDFC profile (bank knowledge as data; same pattern as
before)."""
        return HDFC_PROFILE.first_account(text)


    def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
        """Parse an HDFC statement PDF into a :class:`Statement` of transactions.

        ``password`` is forwarded to pdfplumber for encrypted statements and used
        in-memory only ? never stored or logged.
        """
        path = Path(pdf_path)
        transactions: list[Transaction] = []
        account_number: str | None = None
        with pdfplumber.open(str(path), password=password or "") as pdf:
            for page_no, page in enumerate(pdf.pages):
                if account_number is None and page_no < 2:
                    account_number = _extract_account_number(page.extract_text() or "")
                words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
                body = _table_body(_group_lines(words))
                transactions.extend(extract_transactions_from_lines(body, page_no))
        return Statement(path=path, transactions=transactions,
account_number=account_number)


    def reconcile(statement: Statement, tol: float = BALANCE_TOLERANCE) -> ReconcileReport:
        """Check the running balance via the shared reconciler (closing[i] ==
        closing[i-1] - withdrawal + deposit).

        Spans page breaks (transactions are in document order), so a clean report also
        validates page stitching. Reports counts only ? no amounts.
        """
        entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in
statement.transactions]
        return reconcile_running_balance(entries, tol)


    def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
        """Pure core: read HDFC's SUMMARY block from extracted text lines.

        Layout (stable across statements)::

```

```
            SUMMARY
            Opening Balance  Debit Amount  Credit Amount  Closing Balance
            <opening>        <debitAmt>    <creditAmt>    <closing>
            Debit Count  Credit Count
            <debitCount> <creditCount>

        The values land on their own lines, so we take the first post-SUMMARY line
        carrying ?4 money tokens (the balances) and the first carrying ?2 bare
        integers and no money (the counts).
        """
        start = next((i for i, ln in enumerate(lines) if ln.strip() == "SUMMARY"), None)
        if start is None:
            return None
        balances: list[float] | None = None
        counts: list[int] | None = None
        for ln in lines[start + 1 : start + 8]:
            tokens = ln.split()
            monies = [v for v in (_to_amount(t) for t in tokens) if v is not None]
            ints = [int(t) for t in tokens if t.isdigit()]
            if balances is None and len(monies) >= 4:
                balances = monies[:4]
            elif counts is None and not monies and len(ints) >= 2:
                counts = ints[:2]
        if balances is None and counts is None:
            return None
        bvals: list[float | None] = [*balances] if balances else [None, None, None, None]
        opening, debit_amt, credit_amt, closing = bvals[:4]
        cvals: list[int | None] = [*counts] if counts else [None, None]
        debit_count, credit_count = cvals[:2]
        return StatementSummary(
            opening_balance=opening,
            closing_balance=closing,
            debit_count=debit_count,
            credit_count=credit_count,
            debit_amount=debit_amt,
            credit_amount=credit_amt,
        )


    def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
        """Read the bank's own SUMMARY figures from the statement, or None if absent."""
        with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
            for page in pdf.pages:
                text = page.extract_text() or ""
                if "SUMMARY" in text:
                    summary = _parse_summary_lines(text.splitlines())
                    if summary is not None:
                        return summary
        return None


    def validate_against_summary(
        statement: Statement, summary: StatementSummary, tol: float = BALANCE_TOLERANCE
    ) -> ValidationReport:
        """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.

        Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
        (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
        error/omission, the Dr/Cr counts catch any extra/missing row, and the opening &
closing
        balances catch a missing first/last row.
        """
        rows = [t.to_concept() for t in statement.transactions]
        return verify(rows, summary, tol)

```

```
368
369     def write_csv(statement: Statement, out_path: Path | str) -> Path:
370         """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
371
372         Writes real data ? call only for a local, gitignored destination.
373         """
374         out = Path(out_path)
375         out.parent.mkdir(parents=True, exist_ok=True)
376         with open(out, "w", newline="", encoding="utf-8") as fh:
377             writer = csv.writer(fh)
378             writer.writerow(
379                 [
380                     "date",
381                     "value_date",
382                     "narration",
383                     "ref",
384                     "withdrawal",
385                     "deposit",
386                     "closing_balance",
387                     "page",
388                 ]
389             )
390             for t in statement.transactions:
391                 writer.writerow(
392                     [
393                         t.date,
394                         t.value_date or "",
395                         t.narration,
396                         t.ref or "",
397                         f"{t.withdrawal:.2f}",
398                         f"{t.deposit:.2f}",
399                         f"{t.closing_balance:.2f}",
400                         t.page,
401                     ]
402                 )
403         return out
404
405
406     @register_parser
407     class HDFCParser(StatementParser):
408         """Registered parser for HDFC savings / transaction-account statements."""
409
410         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
411             low = (text or "").lower()
412             # Text evidence is the spelled-out letterhead name or the domain with its TLD ?
the
413             # bare "hdfcbank" token also appears in UPI handles ("?@hdfcbank") inside other
414             # banks' narrations and must not contribute to routing. Sender metadata stays
exempt
415             # (a narration cannot fake the sender).
416             is_hdfc = (
417                 "hdfc bank" in low or "hdfcbank.com" in low or "hdfcbank" in (sender or
"").lower()
418             )
419             is_statement = "savings account details" in low or (
420                 "narration" in low and "closing balance" in low
421             )
422             return is_hdfc and is_statement
423
424         def parse_and_insert(
425             self,
426             pdf_path: Path,
427             conn,
428             doc_id: int,
```

```
            password: str | None = None,
        ) -> int:
            """Persist a parse **only if it reconciles**; otherwise store nothing.

            Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
            balance walk (and the bank's SUMMARY when present). An unreconciled parse is
            quarantined ? status recorded, zero rows persisted ? matching the shared
            savings path (Axis/AU): the ledger never holds rows the oracle didn't endorse.
            Returns the persisted row count.
            """
            try:
                stmt = parse_statement(pdf_path, password=password or "")
            except ParseError:
                raise
            except Exception as exc:
                raise ParseError(f"failed to parse HDFC statement
{redact_filename(pdf_path)}") from exc
            summary = parse_summary(pdf_path, password=password or "")
            ok = (
                validate_against_summary(stmt, summary).ok
                if summary is not None
                else reconcile(stmt).ok
            )
            if not ok:
                dbmod.record_parse(
                    conn,
                    doc_id,
                    confidence=Confidence.UNRECONCILED.value,
                    parser="HDFCParser",
                    parser_version=__version__,
                )
                return 0
            for t in stmt.transactions:
                dbmod.insert_hdfc_transaction(
                    conn,
                    document_id=doc_id,
                    account_number=stmt.account_number,
                    txn_date=normalize_statement_date(t.date),
                    value_date=normalize_statement_date(t.value_date),
                    particulars=t.narration,
                    ref=t.ref,
                    withdrawal=t.withdrawal,
                    deposit=t.deposit,
                    closing_balance=t.closing_balance,
                )
            dbmod.record_parse(
                conn,
                doc_id,
                confidence=Confidence.RECONCILED.value,
                parser="HDFCParser",
                parser_version=__version__,
            )
            return len(stmt.transactions)

```

16. user (2026-06-10T18:00:03.090Z)

```
1       """The **verifier** ? reconciliation as a layout-independent truth oracle.
2
3       Reconciliation proves a parse is **complete and correct without inspecting any sensitive
value**:
4       if a transaction is missed or misread, the arithmetic stops adding up. That is what lets
us trust a
5       parse of a statement we never eyeball, and (later) what makes a probabilistic extractor
safe on
```

```
6       money ? *extraction proposes, reconciliation disposes* (see
7       [`docs/parser-architecture.md`](../../../docs/parser-architecture.md) §2, §5).
8
9       This module grew from a single running-balance walk into the verifier the architecture
calls for:
10
11      * **per-row walk** ? ``reconcile_running_balance``: ``balance[i] == balance[i-1] ? debit
+ credit``.
12      * **total reconcile** ? ``reconcile_total``: ``opening + ?credit ? ?debit == closing``
(works even
13        with no per-row balance, e.g. Axis savings).
14      * **bookend** ? ``verify``: parsed Dr/Cr counts + opening/closing vs the bank's printed
SUMMARY.
15      * **selection** ? ``select_reconciling``: among candidate concept mappings, pick the one
arithmetic
16        endorses (the seam the L2 mapping search plugs into).
17      * **confidence** ? ``Confidence`` + ``BookendReport.confidence``: a recorded verdict,
not an
18        assumption.
19
20      Reports carry counts / booleans / a verdict only ? **never an amount**.
21      """
22
23      from __future__ import annotations
24
25      import re
26      from collections.abc import Sequence
27      from dataclasses import dataclass
28      from enum import StrEnum
29
30      from .concepts import ConceptRow, StatementConcepts
31
32      _CR_DR_SUFFIX = re.compile(r"(?i)\s*(cr|dr)\s*$")
33
34      # Statement money is printed to two decimals, so the smallest REAL discrepancy is one
paisa
35      # (0.01); the walk's float arithmetic accumulates noise many orders of magnitude below
that.
36      # The acceptance tolerance therefore sits at HALF the quantum: genuine float noise
passes,
37      # while an exact one-paisa corruption always fails. (With tol == 0.01 and a strict-
greater
38      # compare, a measured ~40% of exact one-paisa corruptions used to land inside the
tolerance.)
39      BALANCE_TOLERANCE = 0.005
40
41
42      def parse_amount(value: object, *, signed: bool = False) -> float | None:
43          """Parse a statement money cell to a float.
44
45          Returns ``None`` only for a genuinely absent cell (None / empty / ``"-"`` /
46          non-numeric); ``"0.00"`` parses to ``0.0``. This deliberately distinguishes
47          *absent* from *zero* ? unlike ``base.safe_amount``, which maps ``"0.00"`` to
48          ``None`` and would corrupt a real zero balance and the reconciliation math.
49
50          A trailing ``Cr``/``Dr`` marker is interpreted per column kind:
51
52          - ``signed=False`` (default, for **flow** cells): the marker is a *direction* tag;
the
53            magnitude is returned and direction lives in the debit-vs-credit assignment.
54          - ``signed=True`` (for **balance** cells): the marker is a *sign* ? ``Dr`` means the
55            account is overdrawn (negative), ``Cr`` in credit (positive). Stripping the sign
off
56            ``Dr`` balances lets a fully direction-inverted parse of an overdraft statement
57            reconcile, which is the worst possible verifier failure.
```

```
        """
        if value is None:
            return None
        raw = str(value).strip()
        suffix = _CR_DR_SUFFIX.search(raw)
        text = _CR_DR_SUFFIX.sub("", raw).replace(",", "").strip()
        if text in ("", "-"):
            return None
        try:
            amount = float(text)
        except ValueError:
            return None
        if signed and suffix:
            amount = -abs(amount) if suffix.group(1).lower() == "dr" else abs(amount)
        return amount


    @dataclass
    class ReconcileReport:
        """Result of a running-balance walk. Counts only, never an amount."""

        n: int  # rows considered
        compared: int  # consecutive (balance-bearing) pairs actually checked
        breaks: int
        first_break: int | None = None

        @property
        def ok(self) -> bool:
            return self.compared > 0 and self.breaks == 0


    def reconcile_running_balance(
        entries: Sequence[tuple[float | None, float | None, float | None]],
        tol: float = BALANCE_TOLERANCE,
    ) -> ReconcileReport:
        """Verify ``balance[i] == balance[i-1] - debit[i] + credit[i]`` across rows.

        ``entries`` is an ordered list of ``(debit, credit, balance)``; ``debit`` and
        ``credit`` may be ``None`` (treated as 0.0). A row whose ``balance`` is ``None``
        is skipped (can't anchor on it). Because the walk spans the whole statement, a
        clean result also rules out any row missed *between* two balances ? which is the
        failure mode silent ``extract_tables()`` row-drops would otherwise hide.
        """
        breaks = 0
        first_break: int | None = None
        compared = 0
        prev: float | None = None
        for i, (debit, credit, balance) in enumerate(entries):
            if balance is None:
                continue
            if prev is not None:
                expected = prev - (debit or 0.0) + (credit or 0.0)
                compared += 1
                if abs(expected - balance) > tol:
                    breaks += 1
                    if first_break is None:
                        first_break = i
            prev = balance
        return ReconcileReport(
            n=len(entries), compared=compared, breaks=breaks, first_break=first_break
        )


    def _row_entries(
        rows: Sequence[ConceptRow],
```

```
    ) -> list[tuple[float | None, float | None, float | None]]:
        """Project concept rows onto the ``(debit, credit, balance)`` tuples the walk
consumes."""
        return [(r.debit, r.credit, r.balance) for r in rows]


    def reconcile_total(
        rows: Sequence[ConceptRow], opening: float, closing: float, tol: float =
BALANCE_TOLERANCE
    ) -> bool:
        """Verify the layout-independent identity ``opening + ?credit ? ?debit == closing``.

        This is the oracle for statements with **no reliable per-row balance** (e.g. Axis
savings):
        it checks the whole statement against its own opening/closing bookends without
needing each
        row's running balance. ``opening`` and ``closing`` come from the bank's
SUMMARY/header
        (``StatementConcepts``); the caller must have them.
        """
        delta = sum(r.credit or 0.0 for r in rows) - sum(r.debit or 0.0 for r in rows)
        return abs((opening + delta) - closing) <= tol


    class Confidence(StrEnum):
        """Recorded trust verdict for a parsed statement (mirrors ``documents.status``
values).

        ``RECONCILED`` ? the parse passed every available check, so it is trusted.
``UNRECONCILED`` ?
        a check failed (or the document family has checks but none could run); the parse is
suspect
        and is quarantined rather than trusted. ``UNVERIFIED`` ? the document family offers
**no
        arithmetic oracle by design** (e.g. a credit-card bill with no running balance):
rows are
        persisted but explicitly flagged, never presented as trusted. The listing layer
surfaces
        everything that is not cleanly ``RECONCILED``.
        """

        RECONCILED = "reconciled"
        UNRECONCILED = "unreconciled"
        UNVERIFIED = "unverified"


    @dataclass
    class BookendReport:
        """A parse cross-checked against the bank's own SUMMARY. Booleans + counts only.

        ``None`` for a check means the figure wasn't available to verify (not a failure).
``ok``
        requires the per-row balance walk *plus* every available check to pass.
        """

        n: int
        reconciles: bool
        debit_count_ok: bool | None = None
        credit_count_ok: bool | None = None
        opening_ok: bool | None = None
        closing_ok: bool | None = None

        @property
        def ok(self) -> bool:
            checks = [self.debit_count_ok, self.credit_count_ok, self.opening_ok,
```

```
self.closing_ok]
            return self.n > 0 and self.reconciles and all(c is True for c in checks)

        @property
        def confidence(self) -> Confidence:
            return Confidence.RECONCILED if self.ok else Confidence.UNRECONCILED


    def verify(
        rows: Sequence[ConceptRow], concepts: StatementConcepts, tol: float =
BALANCE_TOLERANCE
    ) -> BookendReport:
        """Bookend a parse against the bank's printed SUMMARY: reconciliation + the printed
figures.

        The per-row walk catches any mid-statement error/omission; the Dr/Cr counts catch
any
        extra/missing row; the opening & closing balances catch a missing first/last row.
Each printed
        figure is only checked when present (``None`` ? not checked). The opening balance is
checked by
        implication (``balance[0] ? credit[0] + debit[0]`` should equal the printed
opening).
        """
        reconciles = reconcile_running_balance(_row_entries(rows), tol).ok
        debit_count_ok = credit_count_ok = opening_ok = closing_ok = None
        if concepts.debit_count is not None:
            debit_count_ok = sum(1 for r in rows if (r.debit or 0.0) > 0) ==
concepts.debit_count
        if concepts.credit_count is not None:
            credit_count_ok = sum(1 for r in rows if (r.credit or 0.0) > 0) ==
concepts.credit_count
        if rows and rows[-1].balance is not None and concepts.closing_balance is not None:
            closing_ok = abs(rows[-1].balance - concepts.closing_balance) <= tol
        if rows and rows[0].balance is not None and concepts.opening_balance is not None:
            implied_opening = rows[0].balance - (rows[0].credit or 0.0) + (rows[0].debit or
0.0)
            opening_ok = abs(implied_opening - concepts.opening_balance) <= tol
        return BookendReport(
            n=len(rows),
            reconciles=reconciles,
            debit_count_ok=debit_count_ok,
            credit_count_ok=credit_count_ok,
            opening_ok=opening_ok,
            closing_ok=closing_ok,
        )


    def select_reconciling(
        candidates: Sequence[Sequence[ConceptRow]], tol: float = BALANCE_TOLERANCE
    ) -> int | None:
        """Among candidate concept-row mappings, return the index of the first that
reconciles.

        This is the *selection* half of "extraction proposes, reconciliation disposes": when
the
        extraction ladder can't be sure which column?concept assignment is right, it
proposes several
        and lets the per-row walk pick the correct one. Returns ``None`` if none reconcile.

        The candidate *generation* (the L2 mapping search) is Step 2 of the architecture
plan; this is
        the verifier-side seam it plugs into. Selection here uses the per-row walk, so
candidates need a
        running balance ? balance-less statements select via :func:`reconcile_total` at the
```

```
call site.
        """
        for i, rows in enumerate(candidates):
            if reconcile_running_balance(_row_entries(rows), tol).ok:
                return i
        return None


    __all__ = [
        "BALANCE_TOLERANCE",
        "parse_amount",
        "ReconcileReport",
        "reconcile_running_balance",
        "reconcile_total",
        "Confidence",
        "BookendReport",
        "verify",
        "select_reconciling",
    ]

```

17. user (2026-06-10T18:00:12.294Z)

```
      """Decrypt password-protected statement PDFs (Stage 2).

      Uses pypdf (BSD-3-Clause) ? permissive, pure-Python. Implements the hybrid
      password strategy from DESIGN.md § 8.2: try a candidate dictionary first, then
      fall back to an interactive prompt. The working password is returned so the
      caller can cache it in the OS keychain (muneem.secrets); **it is never logged**.
      """

      from __future__ import annotations

      import shutil
      from collections.abc import Callable, Iterable
      from dataclasses import dataclass
      from pathlib import Path

      import pypdf


      @dataclass
      class UnlockResult:
          ok: bool
          method: str | None  # "plaintext" | "candidate" | "prompt"
          password: str | None  # working password ? do NOT log; None if plaintext/failed
          output_path: Path | None  # decrypted copy, if a destination was given


      def is_encrypted(path: Path | str) -> bool:
          return pypdf.PdfReader(str(path)).is_encrypted


      def password_works(path: Path | str, password: str) -> bool:
          reader = pypdf.PdfReader(str(path))
          if not reader.is_encrypted:
              return True
          try:
              return reader.decrypt(password) != pypdf.PasswordType.NOT_DECRYPTED
          except Exception:
              return False


      def write_decrypted(path: Path | str, password: str | None, dest: Path | str) -> Path:
          """Write a decrypted copy of the PDF to dest (password may be '' for plaintext)."""
```

```
        src = Path(path)
        dest = Path(dest)
        dest.parent.mkdir(parents=True, exist_ok=True)
        reader = pypdf.PdfReader(str(src))
        if reader.is_encrypted:
            reader.decrypt(password or "")
        writer = pypdf.PdfWriter()
        writer.append(reader)
        with open(dest, "wb") as fh:
            writer.write(fh)
        return dest


    def unlock(
        path: Path | str,
        candidates: Iterable[str] = (),
        prompt_fn: Callable[[], str | None] | None = None,
        max_prompts: int = 3,
        dest: Path | str | None = None,
    ) -> UnlockResult:
        """Hybrid unlock: plaintext? -> candidate dictionary -> interactive prompt.

        If dest is given and unlock succeeds, a decrypted copy is written there. The
        working password is returned in the result but must never be logged.
        """
        src = Path(path)
        if not is_encrypted(src):
            out = None
            if dest:
                Path(dest).parent.mkdir(parents=True, exist_ok=True)  # parity with
write_decrypted
                out = Path(shutil.copyfile(src, dest))
            return UnlockResult(True, "plaintext", None, out)

        tried: set[str] = set()
        for pw in candidates:
            if pw in tried:
                continue
            tried.add(pw)
            if password_works(src, pw):
                out = write_decrypted(src, pw, dest) if dest else None
                return UnlockResult(True, "candidate", pw, out)

        if prompt_fn is not None:
            for _ in range(max_prompts):
                entered = prompt_fn()
                if not entered:
                    break
                if password_works(src, entered):
                    out = write_decrypted(src, entered, dest) if dest else None
                    return UnlockResult(True, "prompt", entered, out)

        return UnlockResult(False, None, None, None)

```

18. user (2026-06-10T18:00:12.332Z)

```
    """PDF text extraction (the deterministic, born-digital path).

    Uses **pdfminer.six (MIT)** ? a permissive, commercially-safe, pure-Python text
    extractor. We deliberately do *not* use PyMuPDF/``fitz``: it is **AGPL-3.0**,
    incompatible with muneem's commercial-safety principle (see docs/DEPENDENCIES.md).

    NFKC normalization is applied because the real corpus returns ligatures
    (e.g. U+FB00 '?') that would otherwise break naive matching. Image extraction +
```

```
9       OCR (for codes rasterized into banners, e.g. Swiggy's CTSWIGGYUX) is the image
10      tier and lands with the OCR work (pypdfium2 + Tesseract, both permissive).
11      """
12
13      from __future__ import annotations
14
15      import hashlib
16      import unicodedata
17      from dataclasses import dataclass
18      from pathlib import Path
19
20      from pdfminer.high_level import extract_text
21
22
23      @dataclass
24      class Extracted:
25          """The result of reading a PDF's text layer."""
26
27          path: Path
28          sha256: str
29          text: str
30
31
32      def _normalize(text: str) -> str:
33          """NFKC-normalize so ligatures fold to ASCII (U+FB00 'ff' -> 'ff')."""
34          return unicodedata.normalize("NFKC", text)
35
36
37      def file_sha256(path: Path | str) -> str:
38          """Stream the file and return its hex SHA-256 (used for document dedup)."""
39          digest = hashlib.sha256()
40          with open(path, "rb") as fh:
41              for chunk in iter(lambda: fh.read(65536), b""):
42                  digest.update(chunk)
43          return digest.hexdigest()
44
45
46      def extract(pdf_path: Path | str, password: str = "") -> Extracted:
47          """Extract and NFKC-normalize the full text layer of a PDF.
48
49          ``password`` is forwarded to pdfminer for encrypted statements and used in-memory
only ? it is
50          never stored or logged. The sha256 is over the file bytes, so it is independent of
the password.
51          """
52          path = Path(pdf_path)
53          text = extract_text(str(path), password=password or "")
54          return Extracted(path=path, sha256=file_sha256(path), text=_normalize(text))
55
```

19. user (2026-06-10T18:00:12.467Z)

```
1       from __future__ import annotations
2
3       from .base import StatementParser, safe_amount
4
5       _REGISTRY: list[type[StatementParser]] = []
6
7
8       def register_parser(parser_cls: type[StatementParser]) -> type[StatementParser]:
9           """Decorator to register a parser."""
10          _REGISTRY.append(parser_cls)
11          return parser_cls
12
13
```

```
    def get_parser(
        text: str, sender: str | None = None, subject: str | None = None
    ) -> StatementParser | None:
        """Return an instantiated parser that can handle this document, or None."""
        for parser_cls in _REGISTRY:
            parser = parser_cls()
            if parser.can_handle(text, sender, subject):
                return parser
        return None


    def _autoload_parsers() -> None:
        """Import every parser submodule so its ``@register_parser`` runs.

        Registration is an import *side-effect*, so the registry is only populated for
        modules that have actually been imported. Without this, the production path
        (``from muneem.parsers import get_parser``, as cli.py uses) would leave
        ``_REGISTRY`` EMPTY ? only the tests imported the submodules. Auto-discovery
        fixes that and means new parser modules register themselves with no edit here.
        """
        import importlib
        import pkgutil

        for info in pkgutil.iter_modules(__path__):
            if info.name != "base" and not info.name.startswith("_"):
                importlib.import_module(f"{__name__}.{info.name}")


    _autoload_parsers()

    __all__ = ["StatementParser", "register_parser", "get_parser", "safe_amount"]

```

20. user (2026-06-10T18:01:22.905Z)

```
407:def get_document_by_sha(conn: sqlite3.Connection, sha256: str) -> sqlite3.Row | None:
408-    return conn.execute("SELECT * FROM documents WHERE sha256 = ?", (sha256,)).fetchone()
409-
410-
411:def clear_document_rows(conn: sqlite3.Connection, document_id: int) -> None:
412-    """Remove every per-issuer transaction row tied to a document (pre-re-parse reset).
413-
414-    Used when a previously quarantined/suspect document is re-ingested after a parser fix:
415-    the parse starts from a clean slate under the SAME document id (sha256 stays the dedup
416-    key). Store-nothing parsers leave no rows, so this usually deletes nothing; legacy
417-    residue from before quarantine-parity is cleared too. Provenance upserts on re-parse.
418-    """
419-    for table in (
420-        "axis_transactions",
421-        "axis_cc_transactions",
422-        "au_transactions",
423-        "hdfc_transactions",
424-    ):
425-        conn.execute(f"DELETE FROM {table} WHERE document_id = ?", (document_id,))
426-
427-
428-def _insert_row(conn: sqlite3.Connection, table: str, fields: dict) -> int:
429-    """Internal helper for the simple INSERT pattern used by documents, offers, and bank
txns."""
430-    cols = ", ".join(fields)
431-    placeholders = ", ".join("?" for _ in fields)
432-    cur = conn.execute(
433-        f"INSERT INTO {table} ({cols}) VALUES ({placeholders})",
434-        tuple(fields.values()),
435-    )
```

```
436-    rowid = cur.lastrowid  # always set after a successful INSERT; guard for the type-
checker
--
442:def insert_document(conn: sqlite3.Connection, **fields: object) -> int:
443-    """Insert into documents table. Delegates to the shared helper for DRY."""
444-    return _insert_row(conn, "documents", dict(fields))
445-
446-
447-def record_provenance(
448-    conn: sqlite3.Connection,
449-    *,
450-    document_id: int,
451-    source_type: str,
452-    parser: str | None = None,
453-    parser_version: str | None = None,
454-    mapping_layer: str | None = None,
455-    confidence: str | None = None,
456-) -> None:
457-    """Upsert the provenance row for a parse (one per document ? see DESIGN §8.7).
458-
459-    First-class but *not* per transaction row: every txn table FKs ``documents(id)``, so a
single
460-    provenance record per document is reachable by join. Idempotent on ``document_id`` (a
re-parse
461-    overwrites rather than duplicating).
462-    """
463-    conn.execute(
464-        "INSERT INTO provenance "
465-        "(document_id, source_type, parser, parser_version, mapping_layer, confidence) "
466-        "VALUES (?, ?, ?, ?, ?, ?) "
467-        "ON CONFLICT(document_id) DO UPDATE SET "
--
475:def record_parse(
476-    conn: sqlite3.Connection,
477-    document_id: int,
478-    *,
479-    confidence: str,
480-    parser: str,
481-    source_type: str = "pdf_statement",
482-    parser_version: str | None = None,
483-    mapping_layer: str = "deterministic",
484-) -> None:
485-    """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
parse)."""
486-    conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence, document_id))
487-    record_provenance(
488-        conn,
489-        document_id=document_id,
490-        source_type=source_type,
491-        parser=parser,
492-        parser_version=parser_version,
493-        mapping_layer=mapping_layer,
494-        confidence=confidence,
495-    )
496-
497-
498-def get_provenance(conn: sqlite3.Connection, document_id: int) -> sqlite3.Row | None:
499-    return conn.execute("SELECT * FROM provenance WHERE document_id = ?",
(document_id,)).fetchone()
500-
--
518:def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
519-    return _insert_row(conn, "axis_transactions", fields)
520-
521-
```

```
522:def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
523-    return _insert_row(conn, "axis_cc_transactions", fields)
524-
525-
526-# Per-bank listings carry the parse verdict (documents.status as doc_status) so a consumer
can
527-# tell trusted rows from quarantined/legacy residue without a second query.
528:def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
529-    return conn.execute(
530-        "SELECT t.*, d.status AS doc_status FROM axis_cc_transactions t "
531-        "JOIN documents d ON d.id = t.document_id ORDER BY t.id DESC LIMIT ?",
532-        (limit,),
533-    ).fetchall()
534-
535-
536-def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
537-    return conn.execute(
538-        "SELECT t.*, d.status AS doc_status FROM axis_transactions t "
539-        "JOIN documents d ON d.id = t.document_id ORDER BY t.id DESC LIMIT ?",
540-        (limit,),
541-    ).fetchall()
542-
543-
544:def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
545-    return _insert_row(conn, "hdfc_transactions", fields)
546-
547-
548-def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
549-    return conn.execute(
550-        "SELECT t.*, d.status AS doc_status FROM hdfc_transactions t "
551-        "JOIN documents d ON d.id = t.document_id ORDER BY t.id DESC LIMIT ?",
552-        (limit,),
553-    ).fetchall()
554-
555-
556:def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
557-    return _insert_row(conn, "au_transactions", fields)
558-
559-
560-def list_all_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
561-    """Cross-bank rows from the derived serving view, with each parse's confidence joined
in.
562-
563-    Read-only over ``v_transactions`` (migration v6) ? the flat per-issuer tables remain the
564-    source of truth (DESIGN §8.7).
565-    """
566-    return conn.execute(
567-        """
568-        SELECT v.*, p.confidence
569-        FROM v_transactions v
570-        LEFT JOIN provenance p ON p.document_id = v.document_id
571-        ORDER BY v.document_id DESC
572-        LIMIT ?
573-        """,
574-        (limit,),
575-    ).fetchall()
576-
577-
578:def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) -> list[sqlite3.Row]:
579-    return conn.execute(
580-        "SELECT t.*, d.status AS doc_status FROM au_transactions t "
581-        "JOIN documents d ON d.id = t.document_id ORDER BY t.id DESC LIMIT ?",
582-        (limit,),
583-    ).fetchall()
```

21. user (2026-06-10T18:01:23.396Z)

```
1       [build-system]
2       requires = ["setuptools>=61"]
3       build-backend = "setuptools.build_meta"
4
5       [project]
6       name = "muneem"
7       description = "A local-first personal-finance bookkeeper that ingests financial
documents from email into a consolidated ledger."
8       readme = "README.md"
9       requires-python = ">=3.11"
10      dynamic = ["version"]
11      # Private, single-user tool ? this classifier makes packaging tools refuse to upload it
to PyPI.
12      classifiers = ["Private :: Do Not Upload"]
13      dependencies = [
14          "typer>=0.12",
15          "platformdirs>=4.0",
16          "keyring>=25.0",
17          "pdfminer.six>=20240706",
18          "pypdfium2>=4.30",
19          "pytesseract>=0.3.10",
20          "pillow>=10.0",
21          "pypdf>=4.0",
22          "google-api-python-client>=2.100",
23          "google-auth>=2.30",
24          "google-auth-oauthlib>=1.2",
25          "pdfplumber>=0.11",
26          "sqlcipher3>=0.6",
27          "cryptography>=42",
28      ]
29
30      [project.scripts]
31      muneem = "muneem.cli:app"
32
33      [project.optional-dependencies]
34      outlook = ["msal>=1.0", "requests>=2.0"]
35      dev = [
36          "pytest>=8.0",
37          "pytest-cov>=5.0",
38          "ruff>=0.6",
39          "pyyaml>=6.0",
40          "mypy>=1.8",
41      ]
42
43      [tool.setuptools.dynamic]
44      version = { attr = "muneem.__version__" }
45
46      [tool.setuptools.packages.find]
47      where = ["src"]
48
49      [tool.ruff]
50      line-length = 100
51      src = ["src", "tests"]
52
53      [tool.ruff.lint]
54      select = ["E", "F", "I", "UP", "B"]
55
56      [tool.ruff.format]
57      quote-style = "double"
58      indent-style = "space"
59      skip-magic-trailing-comma = false
60      line-ending = "auto"
61
```

```
62      [tool.ruff.lint.flake8-bugbear]
63      # Typer uses Option()/Argument() as parameter defaults by design.
64      extend-immutable-calls = ["typer.Option", "typer.Argument"]
65
66      [tool.mypy]
67      files = ["src/muneem"]
68      python_version = "3.11"
69      ignore_missing_imports = true   # not all third-party libs ship type stubs
70      check_untyped_defs = true
71      no_implicit_optional = true
72      warn_redundant_casts = true
73      warn_unused_ignores = true
74      warn_unused_configs = true
75
76      # The email-ingestion providers (gmail/imap/outlook) are experimental scaffolding with
77      # NotImplemented methods (docs/CURRENT_STATE.md §1). Type-harden them when the `fetch`
command
78      # lands (ROADMAP B.4); until then don't gate CI on their half-built signatures.
79      [[tool.mypy.overrides]]
80      module = "muneem.ingestion.*"
81      ignore_errors = true
82
83      [tool.pytest.ini_options]
84      # Default runs are offline and deterministic: the ocr/llm/local_statements tiers
85      # (see docs/TESTING.md) are opt-in and excluded here.
86      addopts = "-ra -m 'not ocr and not llm and not local_statements'"
87      testpaths = ["tests"]
88      markers = [
89          "ocr: requires the Tesseract binary (image-OCR tier)",
90          "llm: makes live LLM calls (never in default CI)",
91          "local_statements: reads gitignored real statements (local only)",
92      ]
93
```

22. assistant (2026-06-10T18:04:35.034Z)

Now I have the full picture. Let me check the test environment baseline before writing code.

23. user (2026-06-10T18:04:38.964Z)

```
C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-a0a70ddb1d7197b83\src\muneem\__init__.py
0.11.9 6.12.2
```

24. user (2026-06-10T18:04:42.824Z)

```
..                                                                       [100%]
2 passed in 0.48s
```

25. assistant (2026-06-10T18:04:51.164Z)

Worktree code is what's under test. Now I'll extend synthpdf with multi-page support and ruled-line drawing, then prototype the extract_tables path before writing tests.

26. assistant (2026-06-10T18:05:20.262Z)

```
"""Dependency-free, born-digital PDF generator for regression fixtures (stdlib only).

Layer 1 of the regression-CI plan (ROADMAP § "Planned ? Regression CI") needs **full-pipeline**
fixtures ? real PDFs that go through pdfminer/pdfplumber, not synthetic word-lists that bypass
them.
Rather than add a dependency (reportlab etc.), we emit a minimal valid PDF with **positioned
Helvetica text** (`Tm`/`Tj`), so it stays commercial-safe and CI-portable.

Coordinates: callers think in pdfplumber's top-down `top` (distance from the page top); we
```

```
convert
to PDF's bottom-up baseline. Absolute `top` need not be exact ? parsers key on x-positions,
same-row
grouping, and row order ? but words on one row share a baseline (one `top`) and rows are
separated.
Amount columns are **right-aligned** via real Helvetica metrics so their right edges (`x1`) land
in
the bands the HDFC clusterer / engine expect.

Beyond text, a page can carry stroked **ruling lines** (`m`/`l`/`S` path operators). pdfminer
turns
a stroked two-point path into an ``LTLine``, which is exactly what pdfplumber's default
``extract_tables()`` "lines" strategy keys on ? so a :func:`grid` makes a hand-rolled table
visible
to the cell-matrix extraction path (the Axis credit-card layout). Multi-page statements are
built
with :func:`build_pdf_pages`; :func:`build_pdf` stays the single-page convenience.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from pathlib import Path
from typing import Sequence
```

Helvetica advance widths (per 1000 em). Exact for the chars that matter for right-align (digits / comma / period / minus / space); a reasonable default covers labels (left-align their width is irrelevant to placement).

```
_W = {
    " ": 278,
    "!": 278,
    '"': 355,
    "#": 556,
    "$": 556,
    "%": 889,
    "&": 667,
    "'": 191,
    "(": 333,
    ")": 333,
    "*": 389,
    "+": 584,
    ",": 278,
    "-": 333,
    ".": 278,
    "/": 278,
    "0": 556,
    "1": 556,
    "2": 556,
    "3": 556,
    "4": 556,
    "5": 556,
    "6": 556,
    "7": 556,
    "8": 556,
    "9": 556,
    ":": 278,
    ";": 278,
    "<": 584,
    "=": 584,
    ">": 584,
    "?": 556,
    "@": 1015,
}
```

```
_UPPER = 667  # rough average for A-Z
_LOWER = 528  # rough average for a-z


def helvetica_width(text: str, size: float) -> float:
    """Advance width of ``text`` in Helvetica at ``size`` points."""
    total = 0
    for ch in text:
        if ch in _W:
            total += _W[ch]
        elif ch.isupper():
            total += _UPPER
        else:
            total += _LOWER
    return total / 1000.0 * size


@dataclass
class Placement:
    """One text run: its LEFT edge ``x``, top-down ``top``, and the string."""

    x: float
    top: float
    text: str
    size: float = 9.0


def at(x: float, top: float, text: str, size: float = 9.0) -> Placement:
    """Left-aligned run starting at ``x``."""
    return Placement(x=x, top=top, text=text, size=size)


def right(x1: float, top: float, text: str, size: float = 9.0) -> Placement:
    """Right-aligned run whose right edge lands at ``x1`` (for amount columns)."""
    return Placement(x=x1 - helvetica_width(text, size), top=top, text=text, size=size)


@dataclass
class Line:
    """One stroked straight segment, in the same top-down coordinates as :class:`Placement`.

    pdfminer reports a stroked two-point path as an ``LTLine``, which feeds pdfplumber's
    ``extract_tables()`` "lines" strategy ? draw a full :func:`grid` to make a table ruled.
    """

    x0: float
    top0: float
    x1: float
    top1: float
    width: float = 0.75


def grid(x_edges: Sequence[float], top_edges: Sequence[float]) -> list[Line]:
    """The full ruling of a table: a vertical line per column edge, horizontal per row edge.

    ``extract_tables()``' default strategy intersects these to find cells, so every boundary
    must be drawn (a borderless or semi-ruled table is exactly what it fails on).
    """
    top_lo, top_hi = min(top_edges), max(top_edges)
    x_lo, x_hi = min(x_edges), max(x_edges)
    lines = [Line(x, top_lo, x, top_hi) for x in x_edges]
    lines += [Line(x_lo, t, x_hi, t) for t in top_edges]
    return lines
```

```
@dataclass
class Page:
    """One page's content: positioned text plus any ruling lines."""

    placements: list[Placement]
    lines: list[Line] = field(default_factory=list)


def _esc(text: str) -> str:
    return text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")


def _content_stream(page: Page, height: float) -> bytes:
    """Render one page's lines + text runs as PDF content-stream operators."""
    ops: list[str] = []
    for ln in page.lines:
        # q/Q isolate the line width; top-down -> bottom-up like the text path below.
        ops.append(
            f"q {ln.width:.2f} w {ln.x0:.2f} {height - ln.top0:.2f} m "
            f"{ln.x1:.2f} {height - ln.top1:.2f} l S Q"
        )
    for p in page.placements:
        y = height - p.top  # top-down -> PDF bottom-up baseline
        ops.append(f"BT /F1 {p.size:.2f} Tf 1 0 0 1 {p.x:.2f} {y:.2f} Tm ({_esc(p.text)}) Tj
ET")
    return ("\n".join(ops)).encode("latin-1")


def build_pdf_pages(
    pages: Sequence[Page], path: Path | str, *, width: float = 595.0, height: float = 842.0
) -> Path:
    """Write a multi-page PDF with the given per-page content. Returns the path.

    Hand-rolls a minimal PDF (catalog ? pages ? font ? page/content pairs) with a byte-accurate
    xref table so pdfminer/pdfplumber parse it. Object layout: 1 catalog, 2 page tree, 3 the
shared
    Helvetica font, then each page as a (page, content) object pair.
    """
    page_obj_ids = [4 + 2 * i for i in range(len(pages))]
    kids = " ".join(f"{n} 0 R" for n in page_obj_ids)
    objects: list[bytes] = [
        b"<< /Type /Catalog /Pages 2 0 R >>",
        f"<< /Type /Pages /Kids [{kids}] /Count {len(pages)} >>".encode("latin-1"),
        b"<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>",
    ]
    for i, page in enumerate(pages):
        content = _content_stream(page, height)
        objects.append(
            (
                f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {width:.0f} {height:.0f}] "
                f"/Resources << /Font << /F1 3 0 R >> >> /Contents {page_obj_ids[i] + 1} 0 R >>"
            ).encode("latin-1")
        )
        objects.append(
            b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
b"\nendstream"
        )

    out = bytearray(b"%PDF-1.4\n")
    offsets: list[int] = []
    for i, obj in enumerate(objects, start=1):
        offsets.append(len(out))
        out += f"{i} 0 obj\n".encode() + obj + b"\nendobj\n"

    xref_pos = len(out)
```

```
    count = len(objects) + 1
    out += f"xref\n0 {count}\n".encode()
    out += b"0000000000 65535 f \n"
    for off in offsets:
        out += f"{off:010d} 00000 n \n".encode()
    out += (
        b"trailer\n<< /Size " + str(count).encode() + b" /Root 1 0 R >>\n"
        b"startxref\n" + str(xref_pos).encode() + b"\n%%EOF"
    )

    out_path = Path(path)
    out_path.write_bytes(out)
    return out_path


def build_pdf(
    placements: list[Placement], path: Path | str, *, width: float = 595.0, height: float =
842.0
) -> Path:
    """Write a single-page PDF with the given positioned text. Returns the path."""
    return build_pdf_pages([Page(placements)], path, width=width, height=height)


__all__ = [
    "Placement",
    "Line",
    "Page",
    "at",
    "right",
    "grid",
    "helvetica_width",
    "build_pdf",
    "build_pdf_pages",
]
```

27. user (2026-06-10T18:05:20.283Z)

```
The file
C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-a0a70ddb1d7197b83\tests\synthpdf.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

28. user (2026-06-10T18:05:34.788Z)

```
lines: 15
tables: 1
['Date', 'Ref', 'Transaction Details', 'EMI', 'Intl', 'Cash', 'Pts', 'Merchant Category',
'Amount']
['05/01/2026', '', 'AMAZON PAY INDIA', '', '', '', '', 'ECOMMERCE', '2,499.00 Dr']
['11/01/2026', '', 'SWIGGY BANGALORE', '', '', '', '', 'RESTAURANT', '651.50 Dr']
['18/01/2026', '', 'PAYMENT RECEIVED', '', '', '', '', 'PAYMENT', '3,000.00 Cr']
```

29. assistant (2026-06-10T18:08:18.183Z)

```
"""Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.

Generates *real* PDFs (no committed statements, no secrets) and runs the production pipeline ?
``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert the recorded verdict.
This
covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the synthetic
word-list & table unit tests (which bypass pdfplumber) don't reach. See ROADMAP § "Planned ?
Regression CI for statement parsing" (Layer 1). Always-on, secret-free ? runs in cloud CI.

Corpus breadth (deep-review D1): per-issuer layouts (HDFC, AU, Axis savings), the ruled Axis
credit-card table through ``extract_tables()``, a 2-page statement with a repeated header, a
```

```
bookend-violating statement that must quarantine, and a password-protected statement driven
through the real CLI hybrid-unlock path.
"""

import pypdf
from typer.testing import CliRunner

from muneem import db as dbmod
from muneem import statement_unlock
from muneem.cli import app
from muneem.extract import extract
from muneem.parsers import get_parser
from muneem.unlock import is_encrypted
from synthpdf import Page, Placement, at, build_pdf, build_pdf_pages, grid, right

runner = CliRunner()
```

--- HDFC layout (bespoke coordinate clustering; SUMMARY bookend) ----------

```
def _hdfc_letterhead() -> list[Placement]:
    return [
        at(50, 60, "HDFC BANK"),
        at(50, 72, "Savings Account Details"),
        at(50, 84, "Account Number : 50100012345678"),
    ]


def _hdfc_table_header(top: float = 110) -> list[Placement]:
    """Column header ? must carry "Narration" + a "Closing" token so _table_body locates the
    table; repeated on every page of a real HDFC statement."""
    return [
        at(72, top, "Date"),
        at(175, top, "Narration"),
        at(300, top, "Withdrawals"),
        at(390, top, "Deposits"),
        at(480, top, "Closing"),
        at(515, top, "Balance"),
    ]


def _hdfc_rows(rows: list[tuple[str, str, str, str, str]], top: float = 130) -> list[Placement]:
    """Transaction rows in HDFC geometry: leading date x0?52, narration x0?111, three
    right-aligned amount columns whose right edges land in the clusterer's bands
    (withdrawal <400, deposit <490, balance ?490)."""
    p: list[Placement] = []
    for d, narr, wd, dep, bal in rows:
        p += [
            at(52, top, d),
            at(111, top, narr),
            right(358, top, wd),
            right(443, top, dep),
            right(561, top, bal),
        ]
        top += 15
    return p


def _hdfc_summary(
    figures: str, counts: str, top: float = 200
) -> list[Placement]:
    return [
        at(50, top, "SUMMARY"),
```

```
        at(50, top + 12, "Opening Balance Debit Amount Credit Amount Closing Balance"),
        at(50, top + 24, figures),
        at(50, top + 36, "Debit Count Credit Count"),
        at(50, top + 48, counts),
    ]


def _hdfc_pdf(path):
    """Single-page HDFC statement: three rows that walk (opening 10,000 ? closing 10,000) plus
    the SUMMARY block for the bookend."""
    rows = [
        ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
        ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
        ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
    ]
    p = (
        _hdfc_letterhead()
        + _hdfc_table_header()
        + _hdfc_rows(rows)
        + _hdfc_summary("10,000.00 2,000.00 2,000.00 10,000.00", "2 1")
    )
    return build_pdf(p, path)


def _hdfc_multipage_pdf(path):
    """Two-page HDFC statement: the transaction band splits across pages with the column header
    repeated (as the real statements do), SUMMARY on the last page. The walk and the SUMMARY
    bookend both span the page break, so a clean verdict proves page stitching."""
    page1_rows = [
        ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
        ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
        ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
    ]
    page2_rows = [
        ("12/02/2026", "UPI-BIGBASKET", "800.00", "0.00", "9,200.00"),
        ("15/02/2026", "NEFT-REFUND", "0.00", "1,300.00", "10,500.00"),
    ]
    page1 = _hdfc_letterhead() + _hdfc_table_header() + _hdfc_rows(page1_rows)
    page2 = (
        _hdfc_table_header()
        + _hdfc_rows(page2_rows)
        + _hdfc_summary("10,000.00 2,800.00 3,300.00 10,500.00", "3 2")
    )
    return build_pdf_pages([Page(page1), Page(page2)], path)
```

--- Shared savings layout (the engine-driven savings.py path used by Axis/AU) ----------

```
def _savings_pdf(
    path,
    letterhead: list[str],
    rows: list[tuple[str, str, str, str, str]],
    *,
    opening: str | None = None,
    closing: str | None = None,
):
    """Whitespace-aligned savings table: date | narration | debit | credit | balance in
    well-separated columns ? coordinate clustering ? engine mapping ? per-row reconcile.

    ``letterhead`` carries the routing markers and the account line; ``opening``/``closing``
    print the summary lines the trust oracle anchors a passing walk to
(savings.reconcile_oracle).
    """
```

```
    p: list[Placement] = []
    top = 60
    for line in letterhead:
        p.append(at(50, top, line))
        top += 12
    # header (clustering's date+money filter drops it; here only for realism / text markers)
    p += [
        at(52, 100, "Date"),
        at(110, 100, "Transaction"),
        at(310, 100, "Withdrawal"),
        at(395, 100, "Deposit"),
        at(485, 100, "Balance"),
    ]
    top = 124
    for d, narr, debit, credit, bal in rows:
        line_p = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
        if debit:
            line_p.append(right(340, top, debit))
        if credit:
            line_p.append(right(420, top, credit))
        p += line_p
        top += 15
    if opening is not None:
        p.append(at(50, top + 20, f"Opening Balance : {opening}"))
    if closing is not None:
        p.append(at(50, top + 32, f"Closing Balance : {closing}"))
    return build_pdf(p, path)


_AU_LETTERHEAD = ["AU Small Finance Bank", "Account Number : 1234567890123"]
```

Three rows that walk from an implied opening of 50,000.00.

```
_AU_ROWS = [
    ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
    ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
    ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
]


def _au_savings_pdf(path):
    return _savings_pdf(path, _AU_LETTERHEAD, _AU_ROWS)


def _axis_savings_pdf(path):
    """Axis savings: AXIS_PROFILE dates (dd-mm-yyyy) + the letterhead markers AxisParser routes
    on, with printed Opening/Closing Balance lines consistent with the rows ? exercising the
    bookend-anchored trust oracle on the happy path."""
    rows = [
        ("03-09-2025", "UPI-FLIPKART", "5,000.00", "", "20,000.00"),
        ("10-09-2025", "NEFT-SALARY", "", "40,000.00", "60,000.00"),
        ("18-09-2025", "ATM-WDL", "10,000.00", "", "50,000.00"),
    ]
    return _savings_pdf(
        path,
        ["Axis Bank", "Statement of Account", "Account Number : 912345678901"],
        rows,
        opening="25,000.00",
        closing="50,000.00",
    )


def _au_dropped_first_row_pdf(path):
    """The Axis May-2025 failure class, reproduced at the PDF level: the printed rows are
    internally walk-consistent (the chain 78,800 ? 76,800 holds) but the FIRST transaction of
    the real set is missing, so the first row's implied opening (48,800) contradicts the printed
```

```
    Opening Balance (50,000) of the full set. Only the bookend anchoring can catch this."""
    return _savings_pdf(
        path, _AU_LETTERHEAD, _AU_ROWS[1:], opening="50,000.00", closing="76,800.00"
    )
```

--- Axis credit card (ruled table through pdfplumber extract_tables) ----------

```
def _axis_cc_pdf(path):
    """Axis credit-card statement: a fully *ruled* 9-column table (a synthpdf `grid` of stroked
    lines), because the CC parser reads ``page.extract_tables()`` whose default "lines" strategy
    needs real ruling. Column contract per axis._rows_from_cc_tables: date col 0 (dd/mm/yyyy),
    particulars col 2, merchant category col 7, amount col 8 with a Dr/Cr suffix."""
    x_edges = [40.0, 100.0, 140.0, 280.0, 315.0, 350.0, 385.0, 420.0, 510.0, 565.0]
    tops = [150.0, 170.0, 190.0, 210.0, 230.0]
    p = [
        at(50, 60, "Axis Bank"),
        at(50, 72, "Credit Card Statement"),
        at(50, 84, "Card Number : 4321XXXX9876"),
    ]
    header = ("Date", "Ref", "Transaction Details", "EMI", "Intl", "Cash", "Pts",
              "Merchant Category", "Amount")
    rows = [
        ("05/01/2026", "", "AMAZON PAY INDIA", "", "", "", "", "ECOMMERCE", "2,499.00 Dr"),
        ("11/01/2026", "", "SWIGGY BANGALORE", "", "", "", "", "RESTAURANT", "651.50 Dr"),
        ("18/01/2026", "", "PAYMENT RECEIVED", "", "", "", "", "PAYMENT", "3,000.00 Cr"),
    ]
    for table_row, cells in enumerate((header, *rows)):
        for col, text in enumerate(cells):
            if text:
                p.append(at(x_edges[col] + 3, tops[table_row] + 14, text, size=8))
    return build_pdf_pages([Page(p, lines=grid(x_edges, tops))], path)
```

--- The pipeline under test ----------

```
_TXN_TABLES = ("axis_transactions", "axis_cc_transactions", "au_transactions",
"hdfc_transactions")


def _run_pipeline(pdf) -> tuple[str, int, str, int]:
    """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back the
verdict.

    Returns ``(parser name, rows reported, document status, rows actually stored across every
    per-issuer table)`` ? the stored count lets quarantine tests prove nothing landed.
    """
    ex = extract(pdf)
    parser = get_parser(ex.text)
    assert parser is not None, "no parser routed this synthetic statement"
    with dbmod.connect(":memory:") as conn:
        dbmod.init_db(conn)
        doc_id = dbmod.insert_document(conn, source="file", sha256=ex.sha256, status="parsed")
        n = parser.parse_and_insert(pdf, conn, doc_id)
        status = dbmod.get_document_by_sha(conn, ex.sha256)["status"]
        stored = sum(
            conn.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0] for t in _TXN_TABLES
        )
    return type(parser).__name__, n, status, stored
```

```
def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
    name, n, status, stored = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
    assert name == "HDFCParser"
    assert n == 3, f"expected 3 transactions, got {n}"
    assert status == "reconciled", f"expected reconciled, got {status}"
    assert stored == 3


def test_savings_layout_full_pipeline_reconciles(tmp_path):
    name, n, status, stored = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
    assert name == "AUParser"
    assert n == 3, f"expected 3 transactions, got {n}"
    assert status == "reconciled", f"expected reconciled, got {status}"
    assert stored == 3


def test_axis_savings_layout_full_pipeline_reconciles(tmp_path):
    """Axis savings routes via the title-case letterhead evidence and reconciles through the
    shared engine path, with the printed bookends anchoring the walk."""
    name, n, status, stored = _run_pipeline(_axis_savings_pdf(tmp_path / "axis.pdf"))
    assert name == "AxisParser"
    assert n == 3, f"expected 3 transactions, got {n}"
    assert status == "reconciled", f"expected reconciled, got {status}"
    assert stored == 3


def test_axis_cc_ruled_table_persists_unverified(tmp_path):
    """A CC bill has no running balance, so no oracle can endorse it: rows persist but the
    document is explicitly 'unverified' (Confidence.UNVERIFIED), never silently trusted."""
    name, n, status, stored = _run_pipeline(_axis_cc_pdf(tmp_path / "axis-cc.pdf"))
    assert name == "AxisCreditCardParser"
    assert n == 3, f"expected 3 transactions, got {n}"
    assert status == "unverified", f"expected unverified, got {status}"
    assert stored == 3


def test_multipage_hdfc_reconciles_across_page_break(tmp_path):
    """Rows split across two pages (header repeated) must stitch into one combined set: the
    balance walk and the last-page SUMMARY bookend both span the page break."""
    name, n, status, stored = _run_pipeline(_hdfc_multipage_pdf(tmp_path / "hdfc-2p.pdf"))
    assert name == "HDFCParser"
    assert n == 5, f"expected 5 transactions, got {n}"
    assert status == "reconciled", f"expected reconciled, got {status}"
    assert stored == 5


def test_dropped_first_row_quarantines_at_pdf_level(tmp_path):
    """A statement missing its first transaction still walks internally ? only the printed
    Opening Balance exposes the gap. The pipeline must store NOTHING and record 'unreconciled'
    (the bookend anchoring in savings.reconcile_oracle)."""
    name, n, status, stored = _run_pipeline(_au_dropped_first_row_pdf(tmp_path / "au-gap.pdf"))
    assert name == "AUParser"
    assert n == 0, f"expected 0 rows reported, got {n}"
    assert status == "unreconciled", f"expected unreconciled, got {status}"
    assert stored == 0, f"quarantine must persist nothing, found {stored} row(s)"


def test_encrypted_statement_through_real_cli_hybrid_unlock(tmp_path, monkeypatch):
    """A password-protected statement driven through the REAL `muneem ingest` CLI: pypdf
    encryption on a synthetic AU statement, unlocked by the genuine hybrid path (the keychain
    candidate cache, seeded the same way a successful prompt-unlock caches a password ?
    resolve_password itself is NOT patched). Asserts exit 0 and reconciled rows in the ledger.
    """
    plain = _au_savings_pdf(tmp_path / "plain.pdf")
    enc = tmp_path / "statement.pdf"
```

```
    reader = pypdf.PdfReader(str(plain))
    writer = pypdf.PdfWriter()
    writer.append(reader)
    writer.encrypt("TEST1234")  # synthetic fixture password ? not real material
    with open(enc, "wb") as fh:
        writer.write(fh)
    assert is_encrypted(enc), "the fixture must actually exercise the unlock path"

    # Hermetic: a real password_rules.yaml on the dev machine must not feed candidates.
    monkeypatch.setattr("muneem.password_rules.CONFIG_DIR", tmp_path / "no-config")
    monkeypatch.chdir(tmp_path)
    # Seed the keychain candidate cache through the legitimate seam (conftest isolates the
    # keychain to an in-memory store, so nothing touches the real OS keychain).
    statement_unlock._remember("TEST1234")

    db = tmp_path / "ledger.db"
    result = runner.invoke(
        app, ["ingest", str(enc), "--doc-type", "bank_statement", "--db", str(db), "--no-
prompt"]
    )
    assert result.exit_code == 0, result.output
    assert "[reconciled]" in result.output and "3 transaction(s)" in result.output

    with dbmod.connect_encrypted(db) as conn:
        rows = dbmod.list_au_transactions(conn, limit=10)
    assert len(rows) == 3
    assert all(r["doc_status"] == "reconciled" for r in rows)
```

30. user (2026-06-10T18:08:18.209Z)

```
The file C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-
a0a70ddb1d7197b83\tests\test_regression_synth.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

31. user (2026-06-10T18:08:30.369Z)

```
.......                                                                  [100%]
7 passed in 0.83s
```

32. user (2026-06-10T18:08:41.932Z)

```
The file C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-
a0a70ddb1d7197b83\src\muneem\parsers\savings.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

33. user (2026-06-10T18:08:52.087Z)

```
Exit code 1
>       assert n == 0, f"expected 0 rows reported, got {n}"
E       AssertionError: expected 0 rows reported, got 2
E       assert 2 == 0

tests\test_regression_synth.py:315: AssertionError
========================== short test summary info ==========================
FAILED tests/test_regression_synth.py::test_dropped_first_row_quarantines_at_pdf_level
1 failed in 0.68s
```

34. user (2026-06-10T18:09:01.362Z)

```
The file C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-
a0a70ddb1d7197b83\src\muneem\parsers\savings.py has been updated successfully. (file state is
current in your context ? no need to Read it back)
```

35. assistant (2026-06-10T18:09:06.062Z)

Now the gate: ruff, mypy, full suite with and without coverage.

36. user (2026-06-10T18:09:09.045Z)

```
Exit code 1
E501 Line too long (101 > 100)
   --> tests\synthpdf.py:182:101
    |
180 |         )
181 |         objects.append(
182 |             b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
b"\nendstream"
    |
^
183 |         )
    |

Found 3 errors (2 fixed, 1 remaining).
```

37. user (2026-06-10T18:09:16.484Z)

```
The file
C:\Users\avidu\Projects\muneem\.claude\worktrees\agent-a0a70ddb1d7197b83\tests\synthpdf.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

38. user (2026-06-10T18:09:37.311Z)

```
All checks passed!
1 file reformatted, 80 files left unchanged
Success: no issues found in 34 source files
```

39. user (2026-06-10T18:09:54.576Z)

```
src\muneem\parsers\savings.py        133      6    95%
src\muneem\parsers\validation.py      92      2    98%
src\muneem\password_rules.py          41      1    98%
src\muneem\recovery.py                63      0   100%
src\muneem\redaction.py               23      0   100%
src\muneem\secrets.py                 20      0   100%
src\muneem\statement_unlock.py        45      6    87%
src\muneem\unlock.py                  59      5    92%
-----------------------------------------------------
TOTAL                               2158    250    88%
Required test coverage of 80% reached. Total coverage: 88.42%
514 passed, 6 deselected in 7.25s
```

40. user (2026-06-10T18:10:07.652Z)

```
.......................................................................... [ 98%]
..........                                                               [100%]
514 passed, 6 deselected in 4.48s
```

41. assistant (2026-06-10T18:10:15.490Z)

```
All green: 514 passed, coverage 88.42%. Now ship.
```

42. user (2026-06-10T18:10:18.179Z)

```
 M tests/synthpdf.py
 M tests/test_regression_synth.py
 tests/synthpdf.py              | 118 ++++++++++---
 tests/test_regression_synth.py | 340 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++------
 2 files changed, 392 insertions(+), 66 deletions(-)
```

43. user (2026-06-10T18:10:33.865Z)

```
Switched to a new branch 'test/synthetic-corpus-breadth'
[test/synthetic-corpus-breadth b01a71d] test: broaden the synthetic regression corpus (deep-
review D1)
 2 files changed, 392 insertions(+), 66 deletions(-)
```

44. user (2026-06-10T18:10:43.876Z)

```
branch 'test/synthetic-corpus-breadth' set up to track 'origin/test/synthetic-corpus-breadth'.
remote:
remote: Create a pull request for 'test/synthetic-corpus-breadth' on GitHub by visiting:

remote:      https://github.com/avidullu/muneem/pull/new/test/synthetic-corpus-breadth

remote:
To https://github.com/avidullu/muneem.git
 * [new branch]      test/synthetic-corpus-breadth -> test/synthetic-corpus-breadth
```

45. user (2026-06-10T18:11:12.024Z)

```
https://github.com/avidullu/muneem/pull/70
```

46. assistant (2026-06-10T18:11:29.132Z)

PR: **https://github.com/avidullu/muneem/pull/70** (branch `test/synthetic-corpus-breadth`, not merged)

Deliverables ? all 5 shipped, none blocked:

1. **Axis savings** ? born-digital fixture routed via title-case letterhead ("Axis Bank" + "Statement of Account") to `AxisParser`, through coordinate clustering + L2 engine search, reconciled with printed Opening/Closing Balance lines anchoring the walk. 3 rows persisted, `reconciled`.
2. **Axis CC** ? true PDF route through `extract_tables()` worked: added `Line`/`grid` primitives to `tests/synthpdf.py` that stroke `m`/`l`/`S` paths (pdfminer ? `LTLine` ? pdfplumber "lines" strategy). The 9-column ruled table extracts cleanly; 3 rows persist with status `unverified`. No fallback seam needed.
3. **Multi-page HDFC** ? 3+2 rows across 2 pages, header repeated, SUMMARY on page 2; walk + bookend span the page break. 5 rows, `reconciled`. (`synthpdf` gained `build_pdf_pages`/`Page`; `build_pdf` unchanged for existing callers.)
4. **Dropped-row quarantine** ? AU statement, internally walk-consistent but missing the first transaction vs the printed Opening Balance: 0 rows stored, `unreconciled`. Verified by mutation: disabling the opening-anchor in `savings.reconcile_oracle` made it wrongly persist 2 rows ? the test exercises exactly the new bookend anchoring (mutation reverted).
5. **Encrypted-PDF CLI** ? pypdf RC4-128 encryption on a synthetic AU statement, driven through real `muneem ingest --doc-type bank_statement --no-prompt` via `CliRunner`. `resolve_password` not patched; candidate seeded via `statement_unlock._remember` against the conftest in-memory keychain. Exit 0, `[reconciled]`, 3 rows read back from the encrypted ledger.

Test counts: 7 tests in `tests/test_regression_synth.py` (up from 2); full default suite 514 passed, 6 deselected; coverage 88.42% (gate 80). `ruff check`, `ruff format`, `mypy src` all clean. Verified the worktree src (not the editable install) was under test via the mutation check.

Caveats: the CLI test pins `muneem.password_rules.CONFIG_DIR` to an empty tmp dir + `chdir(tmp_path)` for hermeticity (a real `password_rules.yaml` on the dev machine would otherwise add unlock candidates ? harmless, since the seeded cache is tried first, but nondeterministic). `_run_pipeline`'s return shape changed (now includes stored-row count), updating the two pre-existing tests in the same file. CC encryption uses pypdf's default RC4-128 ? decryptable by pdfminer/pdfplumber with no extra backend.